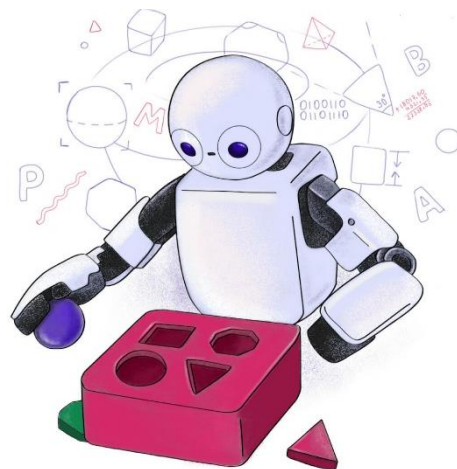
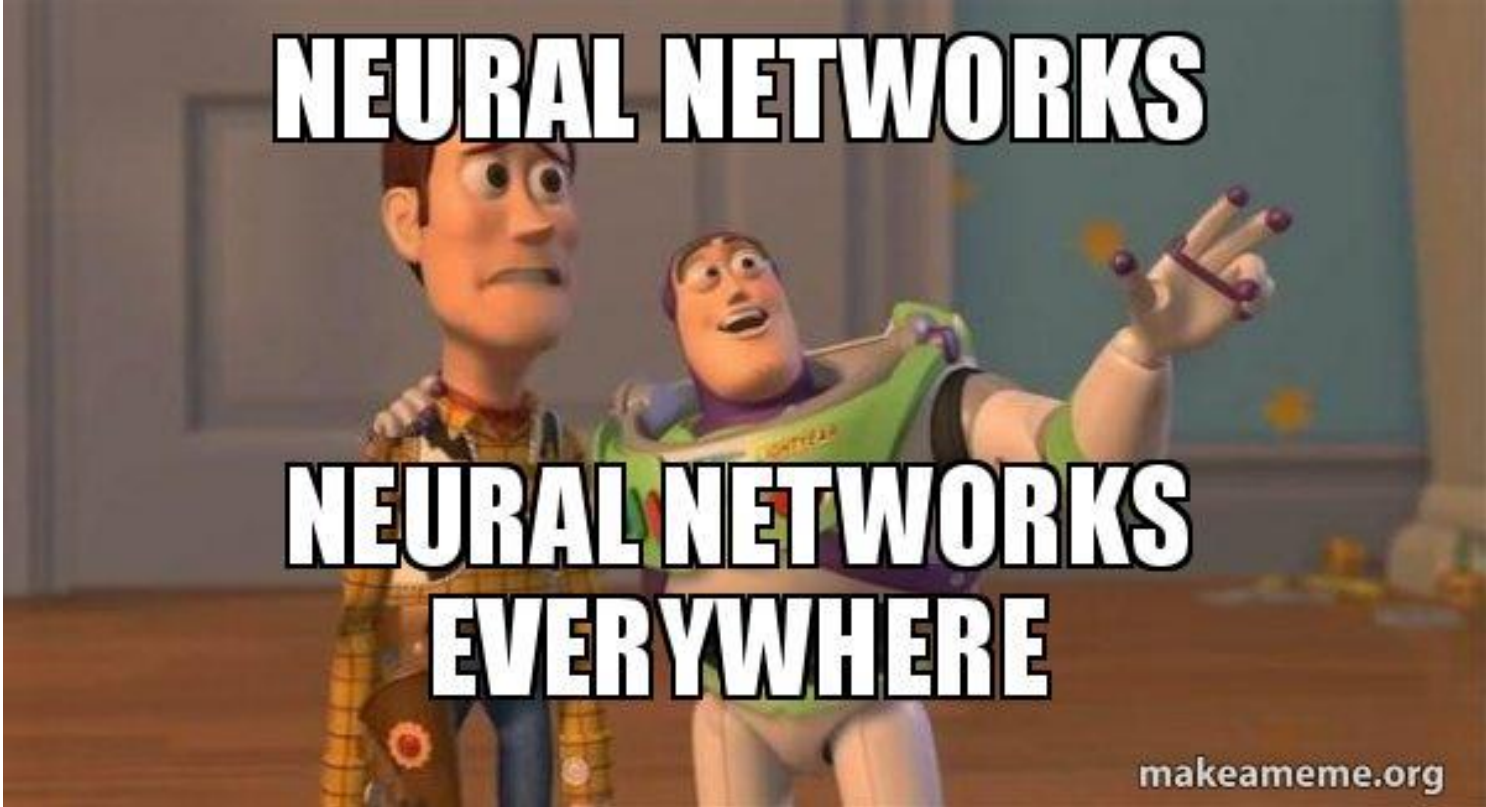


C24 – Inteligência Artificial: *Redes Multilayer Perceptron*



Inatel

Felipe A. P. de Figueiredo
felipe.figueiredo@Inatel.br



NEURAL NETWORKS

**NEURAL NETWORKS
EVERYWHERE**

makeameme.org

São os modelos de ML mais usados atualmente.

ChatGPT

FaceID

Filtros do Instagram

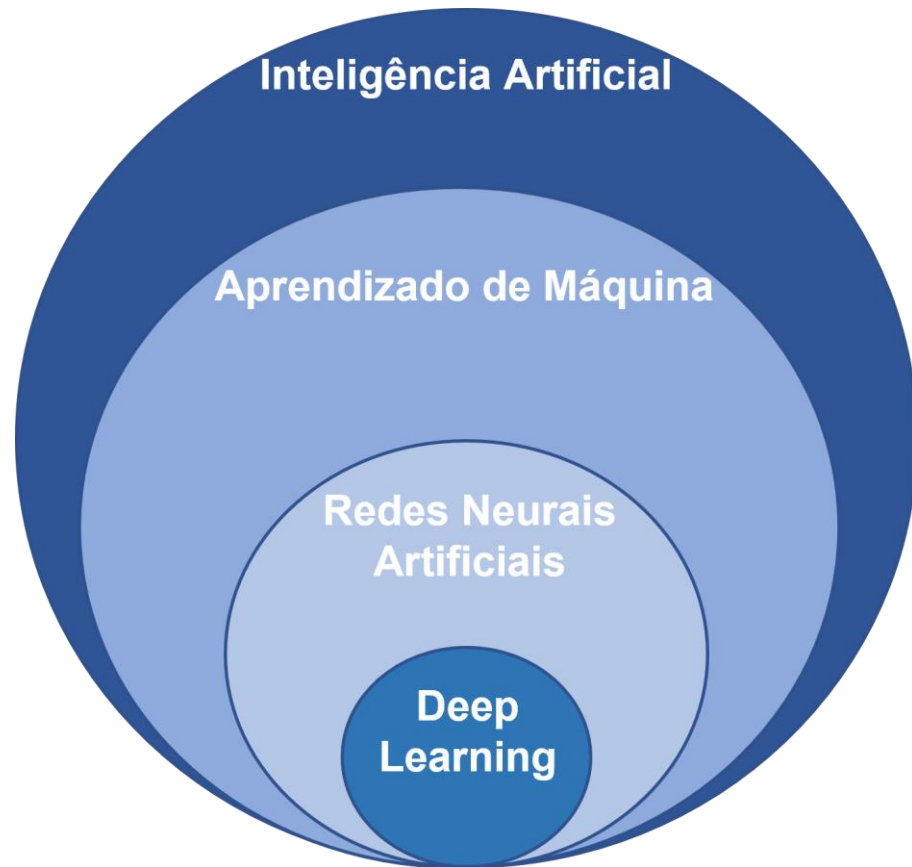
Alexa

Recomendação (Netflix, Spotify)

Google tradutor

Filtros de spam

Redes neurais artificiais



Subárea do aprendizado de máquina.

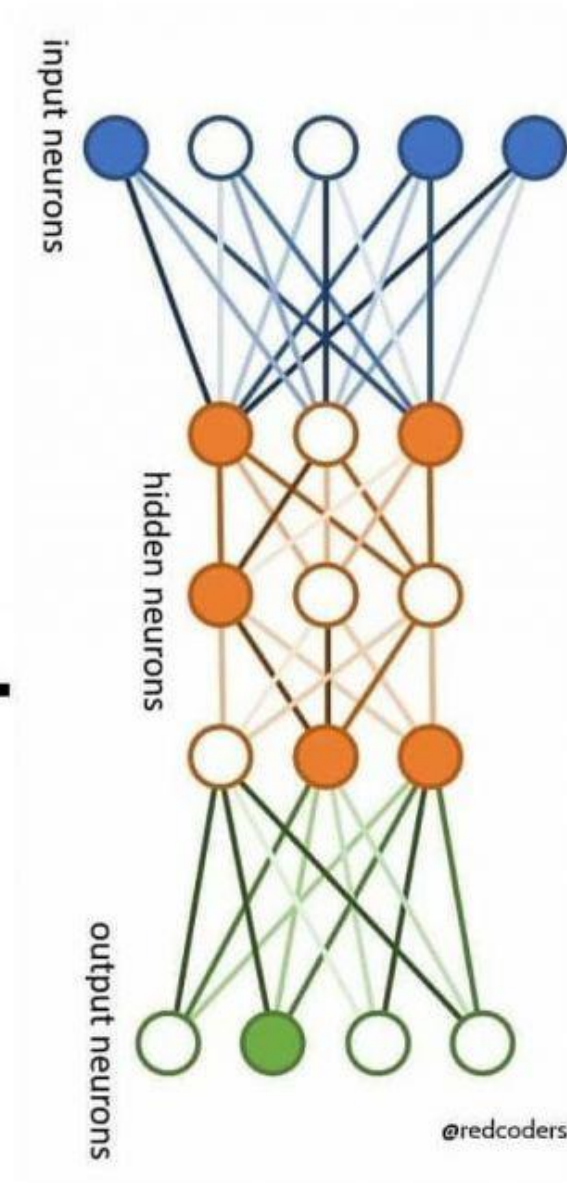
Ela engloba o aprendizado profundo (*deep learning*).

O que é uma rede neural
artificial?

**THIS IS A NEURAL
NETWORK.**

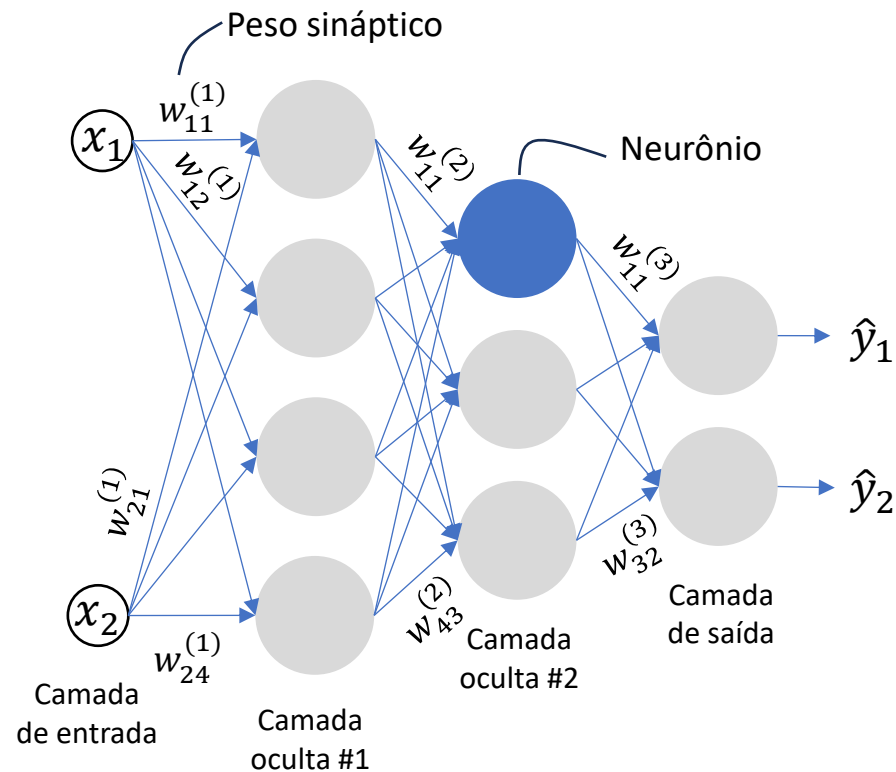
**IT MAKES MISTAKES.
IT LEARNS FROM THEM.**

**BE LIKE A NEURAL
NETWORK.**



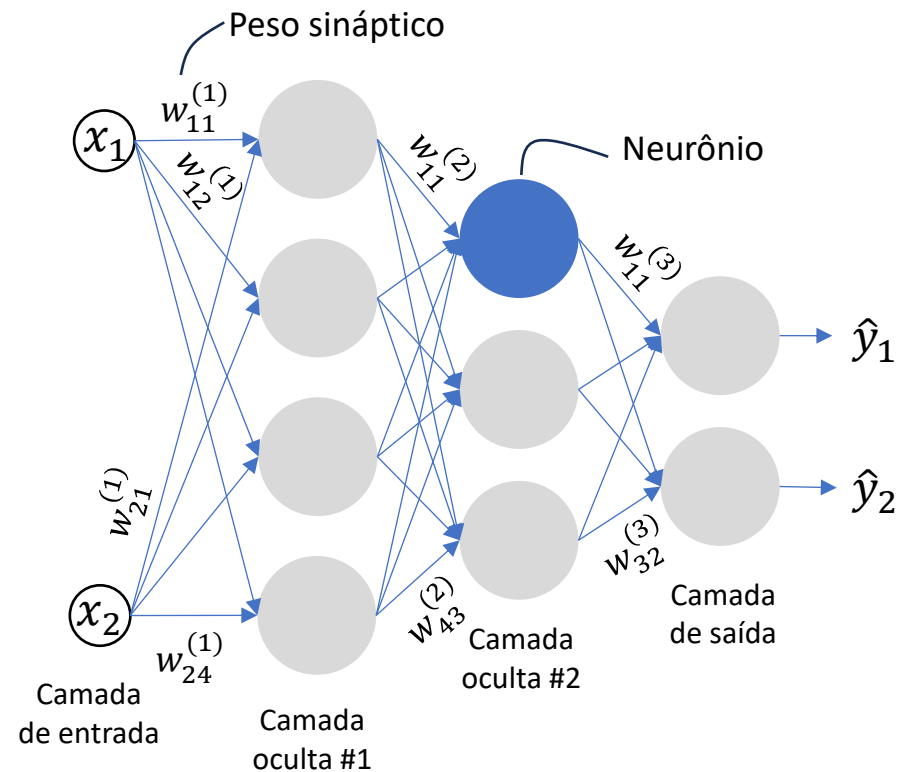
Modelo com múltiplas
entradas, com uma ou mais
camadas, onde cada uma
delas contém vários
neurônios interconectados
que geram uma ou mais
saídas.

O que é uma rede neural artificial?



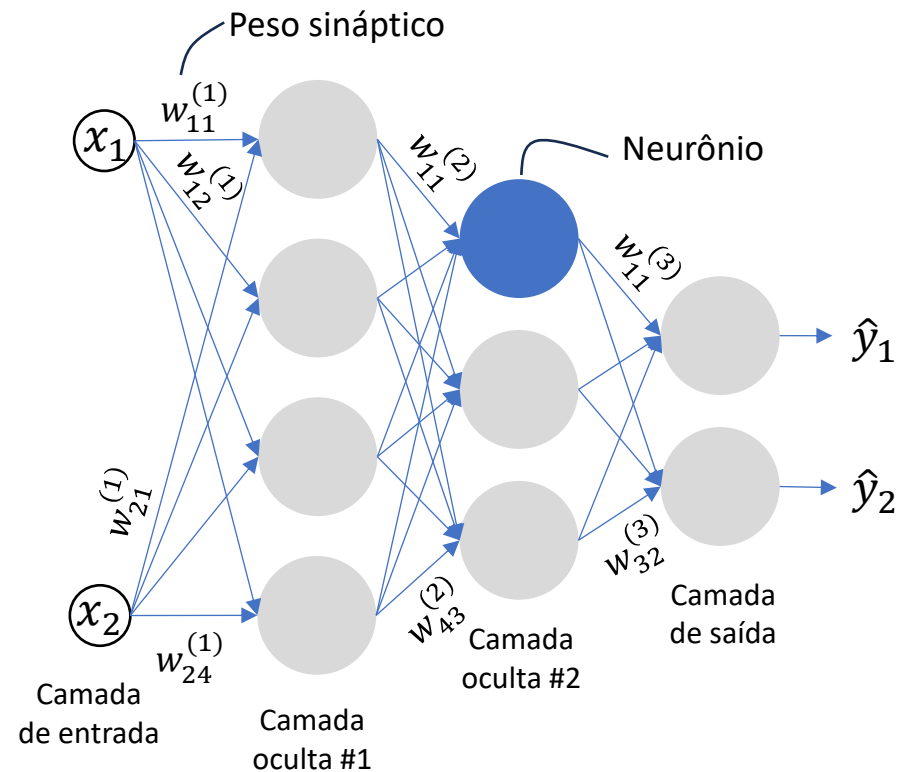
- **Redes neurais artificiais** são modelos matemáticos **inspirados** pelo funcionamento do cérebro dos animais.
- Elas são capazes de realizar **tarefas de aprendizado de máquina** com grande versatilidade e eficácia.
 - regressão,
 - classificação,
 - clusterização,
 - detecção e segmentação,
 - rastreamento de objetos,
 - etc.

O que é uma rede neural artificial?



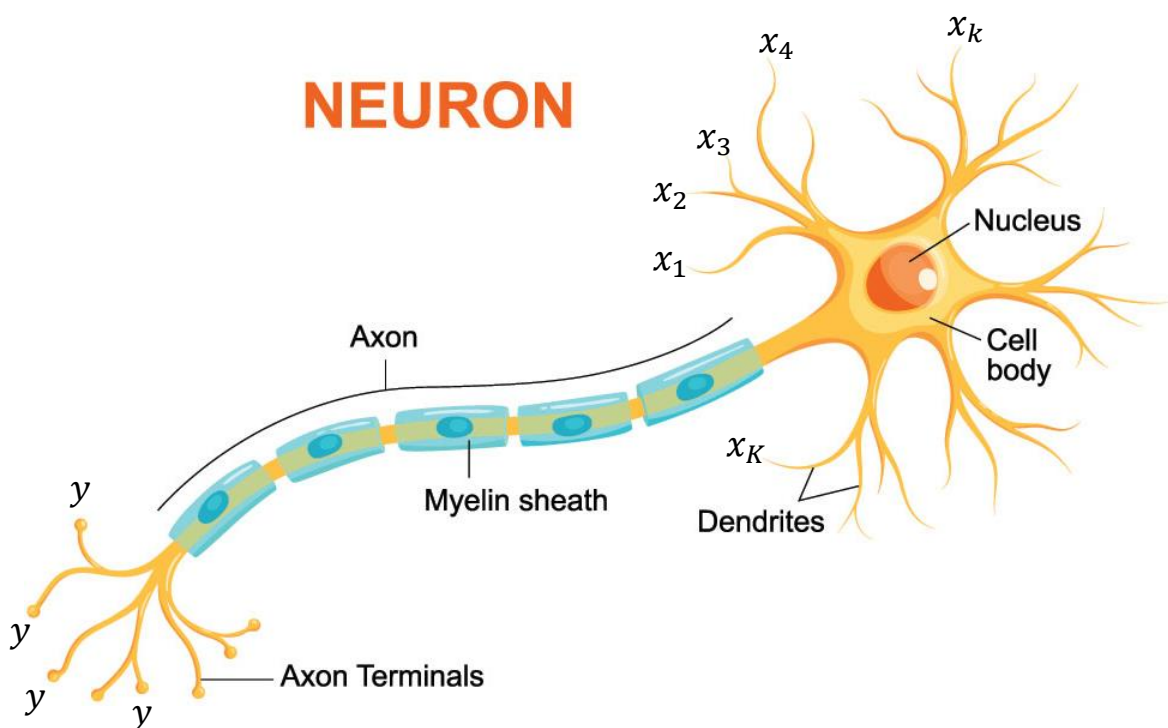
- RNAs são representadas como
 - sistemas de **neurônios (ou nós) interconectados**,
 - organizados em **camadas**,
 - que **aprendem, a partir de dados**, uma **função que relaciona entradas às saídas**.

O que é uma rede neural artificial?



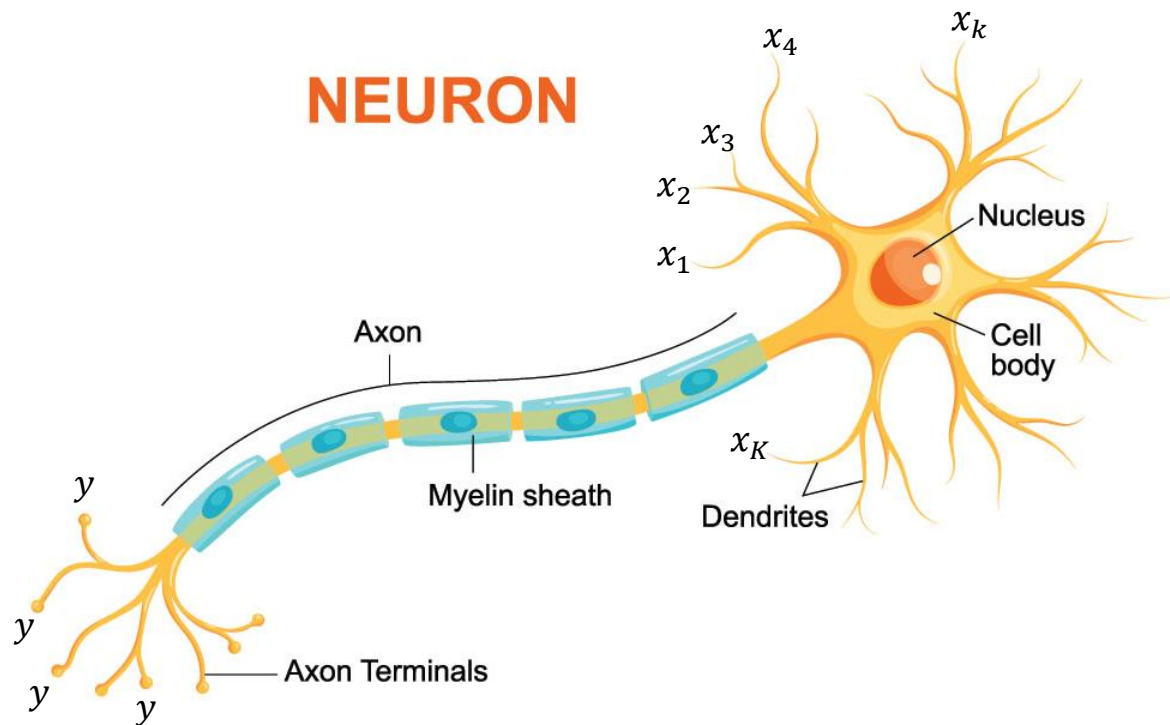
- Antes de entendermos como funciona uma rede neural, **precisamos primeiro lembrar o princípio básico de funcionamento um neurônio biológico** e como ele é usado para modelar um **neurônio artificial**.

Neurônio biológico



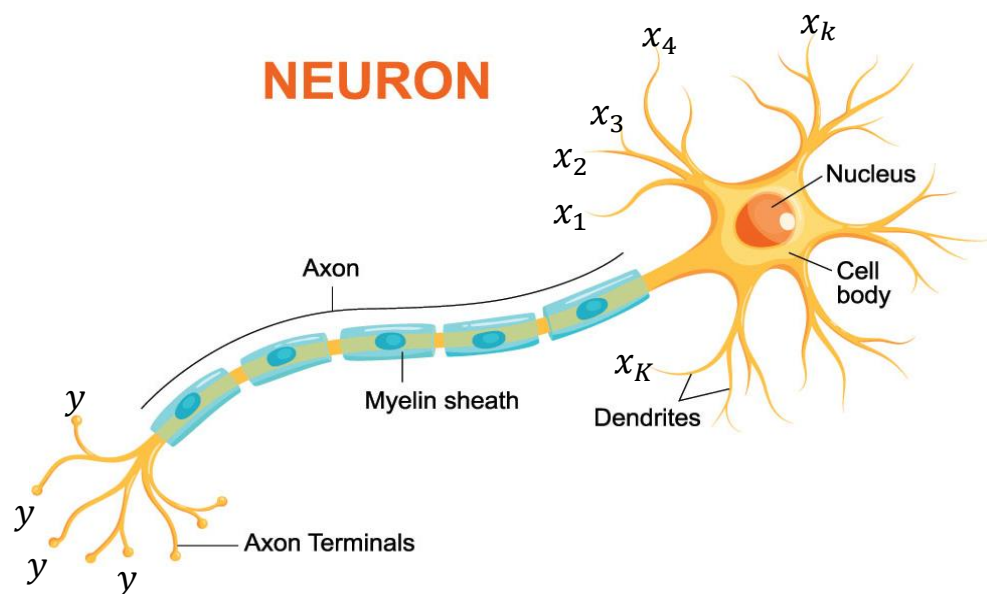
- São **células** que possuem **mecanismos eletroquímicos** para realizar a **transmissão de sinais elétricos** (i.e., informações) **ao longo do sistema nervoso**.
- Têm três partes fundamentais:
 - os **dendritos**,
 - o **axônio com seus terminais**
 - e o **corpo celular**, também chamado de **soma**.

Neurônio biológico



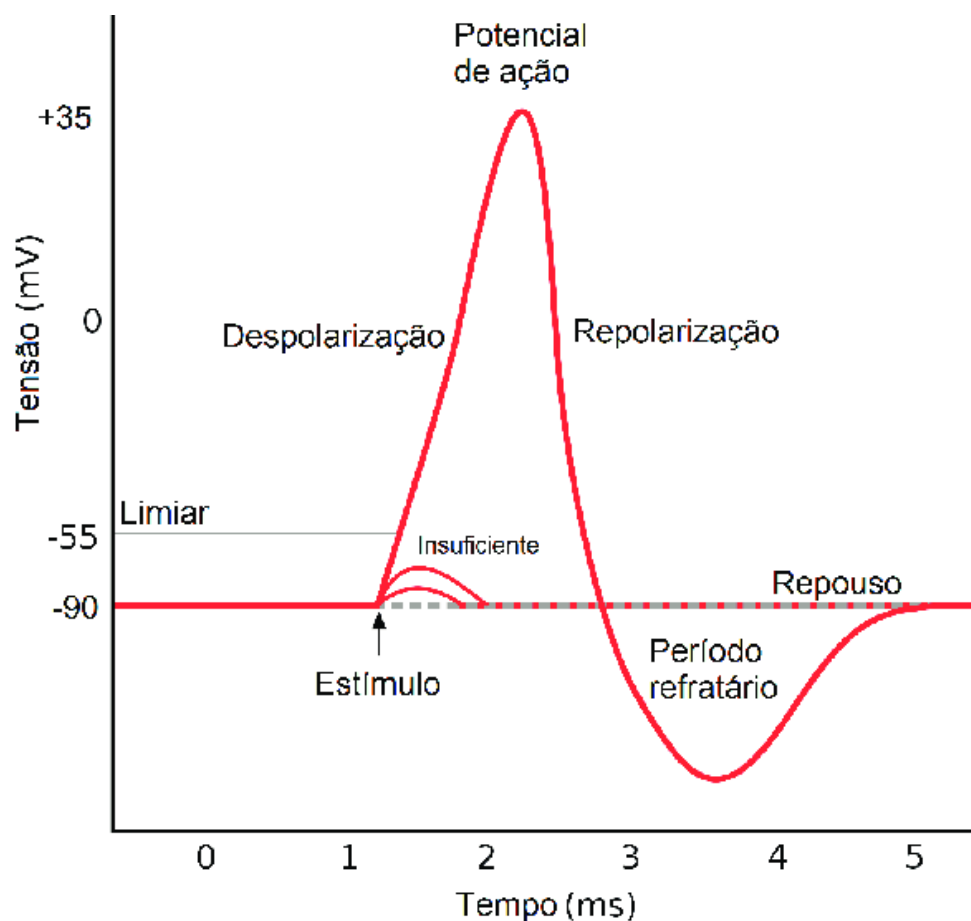
- Os **dendritos** recebem **estímulos** vindos de outros neurônios e os levam em direção ao **corpo celular**.
- O **corpo celular** realiza a **integração** dos estímulos e gera **impulsos** caso um limiar de ativação seja ultrapassado.
- O **axônio** envia **impulsos** a outros neurônios através de seus **terminais**.

Neurônio biológico



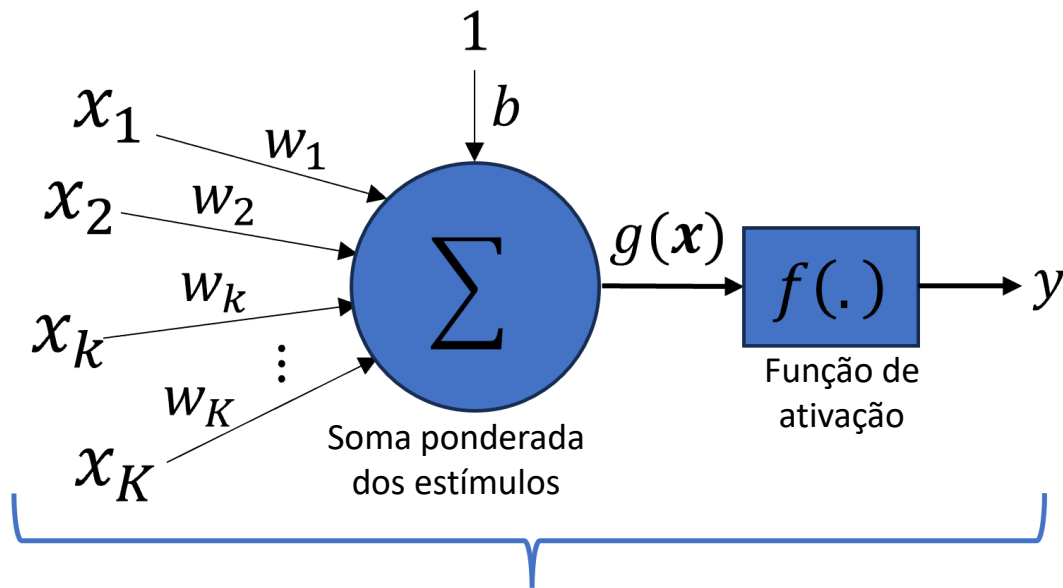
- Os neurônios se comunicam uns com os outros através das **sinapses**.
- **Sinapses** são os **pontos de contato** entre os **dendritos** de um neurônio e os **terminais do axônio** de outros neurônios.
- As sinapses podem ser fortes ou fracas:
 - **Forte**: a informação de um neurônio é importante para a ativação do próximo.
 - **Fraca**: a informação de um neurônio não é tão importante para a ativação do próximo.

Neurônio biológico



- Em termos bem simples, o **neurônio** funciona da seguinte forma:
 - Ele **recebe estímulos elétricos** através dos dendritos.
 - Esses **estímulos são integrados** no corpo celular (soma).
 - Se a **integração dos estímulos** exceder um certo **limiar de ativação**, o **neurônio** gera um **impulso** (ou **potencial de ação**) que é enviado pelos terminais do axônio a outros neurônios através das **sinapses**.

Neurônio artificial

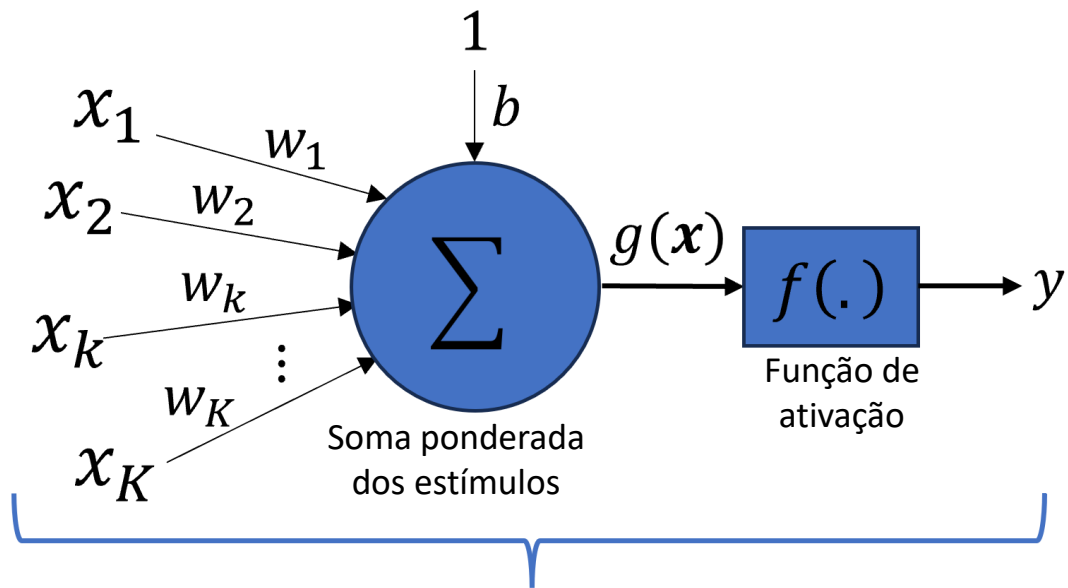


$$y = f(g(\mathbf{x})) = f\left(\left(\sum_{i=1}^K w_i x_i\right) + b\right)$$

- Baseado no entendimento do funcionamento do neurônio biológico, a partir de **meados da década de 40**, pesquisadores propuseram o **modelo matemático ao lado**.
 - É uma **simplificação e abstração** do funcionamento do neurônio biológico.

Qual modelo de ML aprendido anteriormente se assemelha ao neurônio artificial?

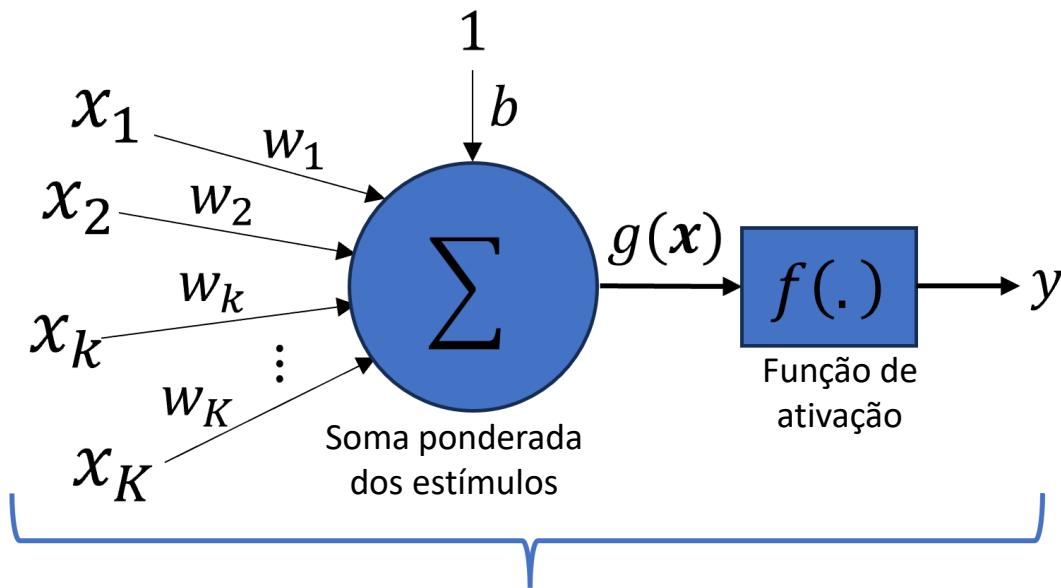
Neurônio artificial



$$y = f(g(\mathbf{x})) = f\left(\left(\sum_{i=1}^K w_i x_i\right) + b\right)$$

- O neurônio se assemelha ao **regressor logístico**.
- Os **estímulos**, $x_k, k = 1, \dots, K$, são **multiplicados** pelos **pesos sinápticos**, $w_k, k = 1, \dots, K$, somados ao **peso de bias**, b , e o resultado, $g(\mathbf{x})$, é passado através de uma **função de ativação**, $f(\cdot)$.

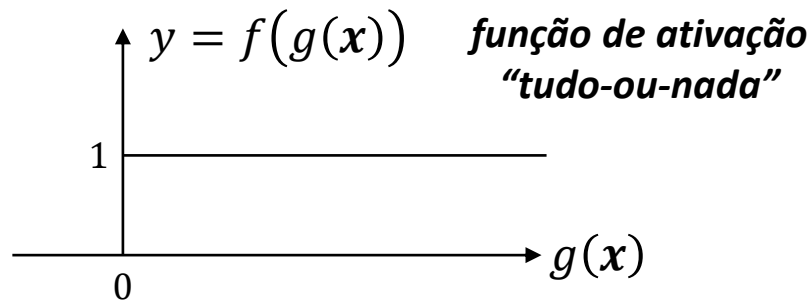
Neurônio artificial



$$y = f(g(\mathbf{x})) = f\left(\left(\sum_{i=1}^K w_i x_i\right) + b\right)$$

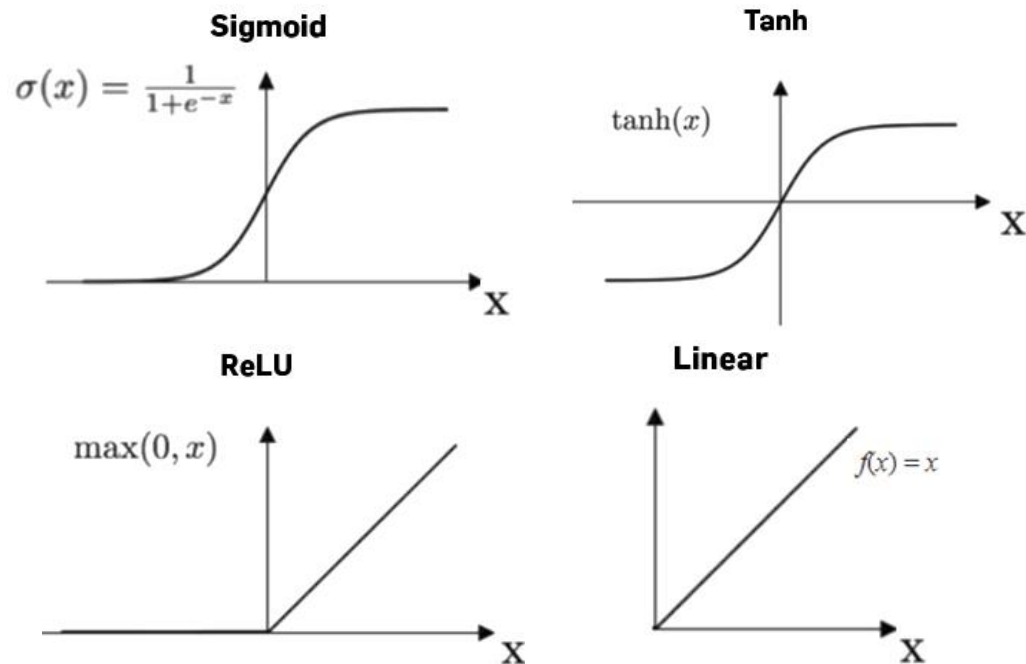
- No contexto de redes neurais os estímulos, $x_k, k = 1, \dots, K$, são chamados de **atributos**.
- O **peso de bias**, b , permite **ajustar o limiar de ativação** (ou ponto de disparo) da função de ativação **do neurônio**.
- Os **pesos sinápticos** w_k dão a **força de cada conexão** e se ela é **excitatória ou inibitória**.
- A média ponderada dos atributos, $g(\mathbf{x})$, é chamada de **ativação**.

Neurônio artificial



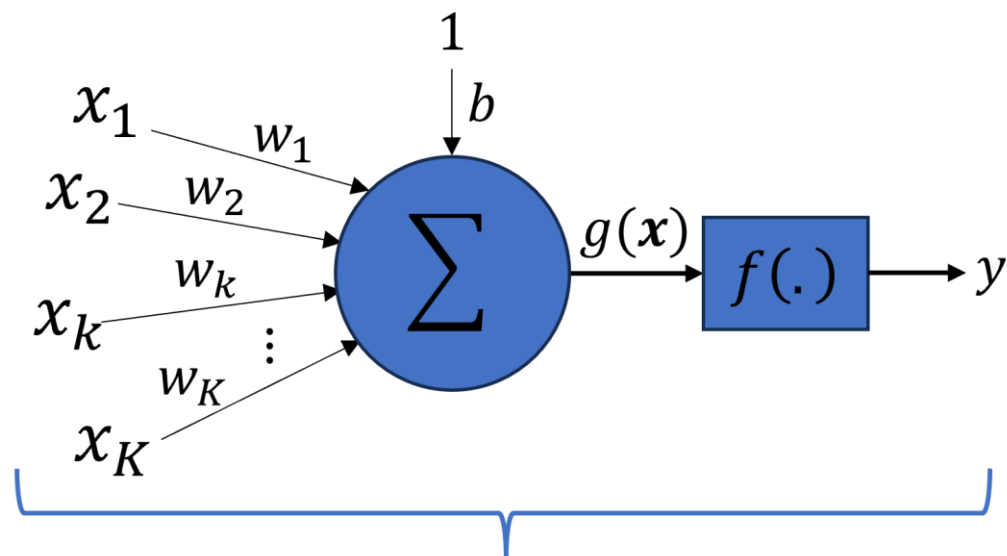
- No início dos estudos dos neurônios, a **função de ativação** era do tipo **tudo-ou-nada**, i.e., ativava ou não.
 - Em outras palavras, era um degrau unitário.
- Entretanto, ela não é usada porque sua **derivada é zero** em quase todos os pontos.
- Isso **impede o ajuste dos pesos** pelo algoritmo de **gradiente descendente**.

Neurônio artificial



- Assim, com o decorrer do tempo, funções de ativação **contínuas e deriváveis** foram propostas:
 - sigmoide (ou logística),
 - tangente hiperbólica,
 - linear,
 - relu e suas variantes,
 - etc.
- **OBS.:** A função “linear” é usada, em geral, apenas na saída de redes neurais para regressão.

Neurônio artificial

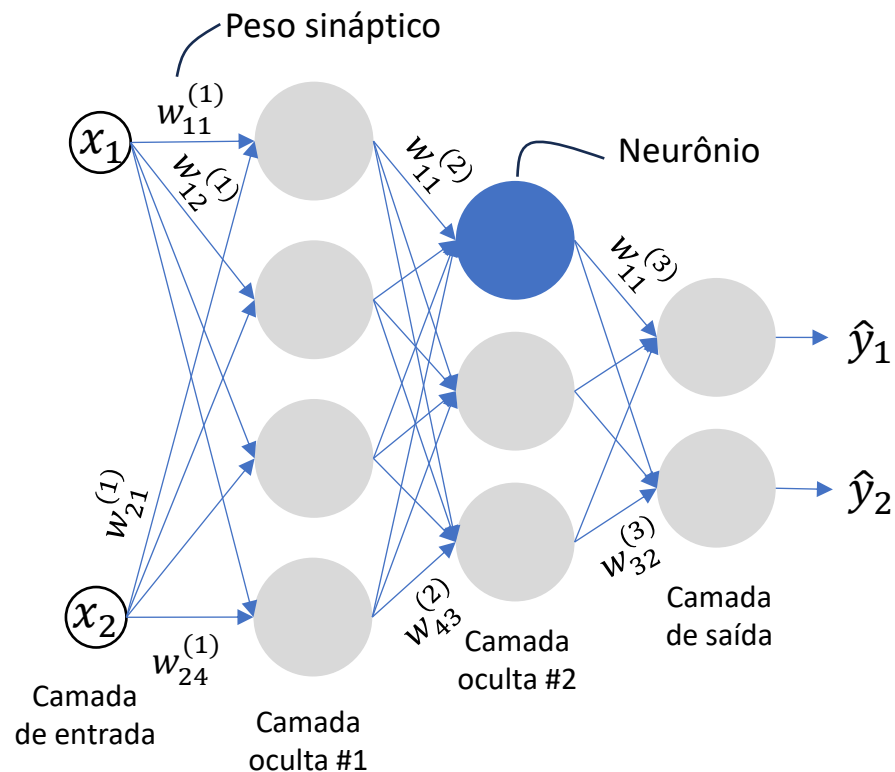


$$y = f \left(\left(\sum_{i=1}^K w_i x_i \right) + b \right)$$

Pesos ou
parâmetros

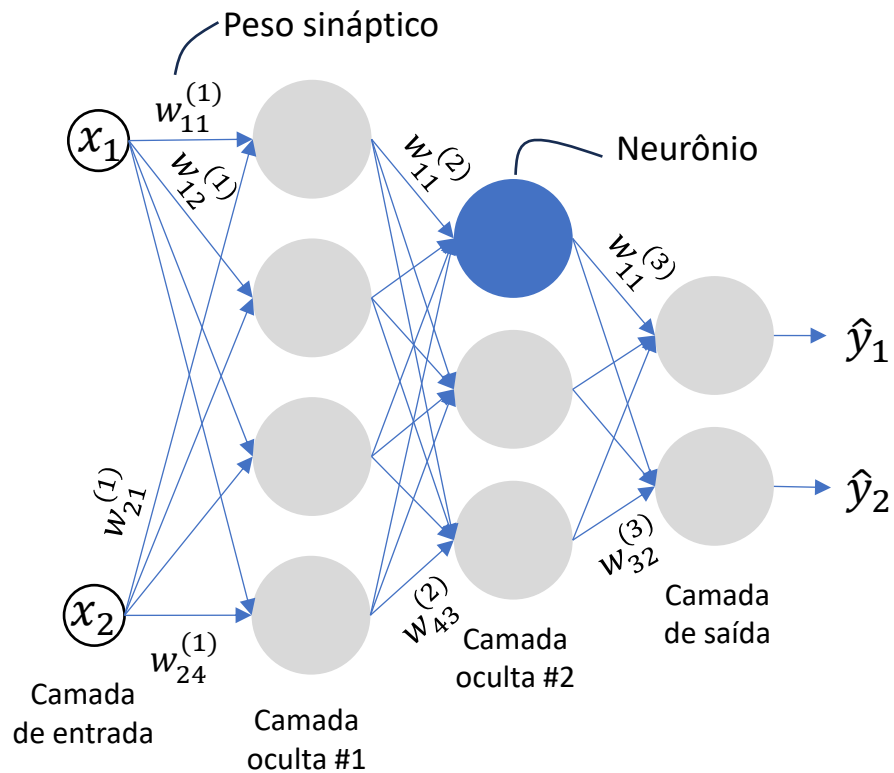
- O **objetivo** do **treinamento** de uma rede neural artificial é **encontrar os valores ideais dos pesos** (bias e sinápticos) **de todos os neurônios** de forma que a rede **resolva uma tarefa específica**, como, por exemplo, a classificação de imagens ou a previsão do preço de casas.
- Ou seja, em problemas **supervisionados**, o objetivo é **minimizar uma função de erro**.

Neurônio artificial



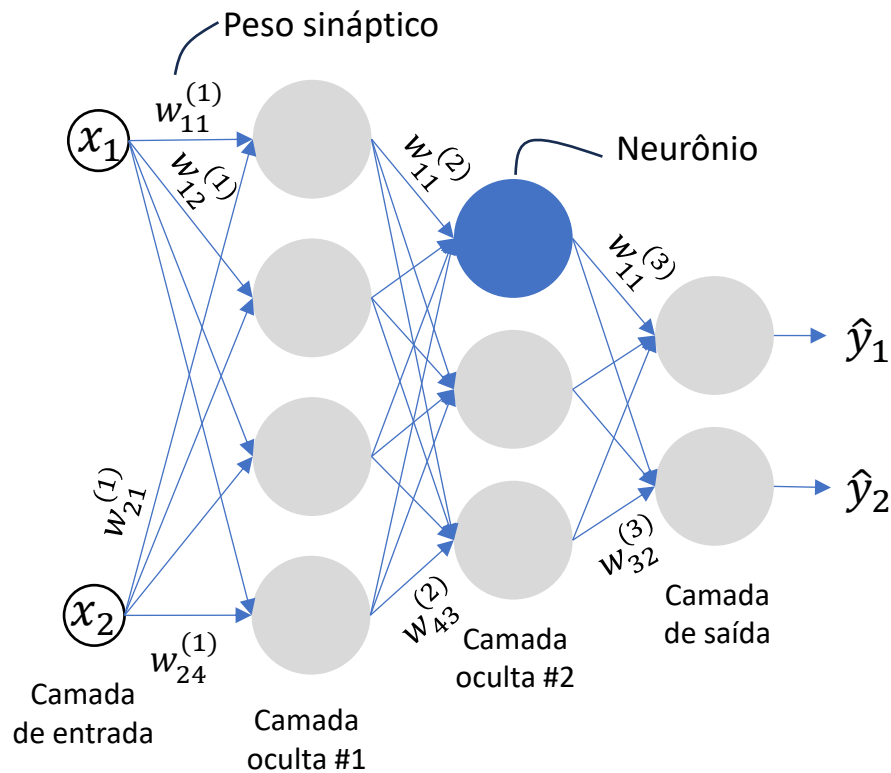
- Os pesos são ajustados através de um **algoritmo de otimização iterativo** chamado de **retropropagação do erro**.
- O algoritmo apresenta ao modelo as **entradas** e tem como **objetivo encontrar os pesos que minimizam o erro** entre a **saída da rede** e os **rótulos (labels)**.

Redes neurais artificiais



- As RNAs são formadas por uma camada de entrada, zero ou mais camadas intermediárias e uma camada de saída.
- A camada de entrada é o ponto de transferência dos atributos para dentro da rede.
 - Essa camada **não tem pesos**.
 - Ela pode ser usada para aplicar **transformações de dimensão** aos dados de entrada.
- Cada camada, exceto a de entrada, possui um ou mais neurônios.

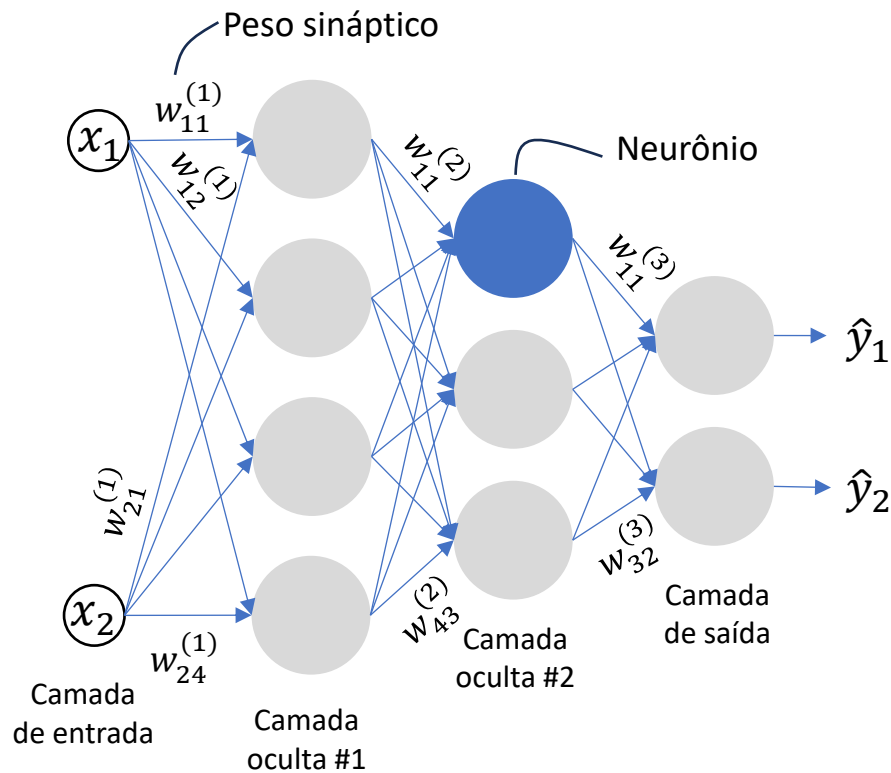
Redes neurais artificiais



- As camadas intermediárias são também chamadas de **ocultas** ou **escondidas**.
- As camadas ocultas permitem:
 - criar **representações intermediárias**.
 - extrair **características relevantes**.
- O primeiro tipo de rede neural proposto foi o **perceptron de múltiplas camadas** (do inglês, *Multilayer Perceptron* - MLP).
 - Rede proposta em 1958 por Frank Rosenblatt.

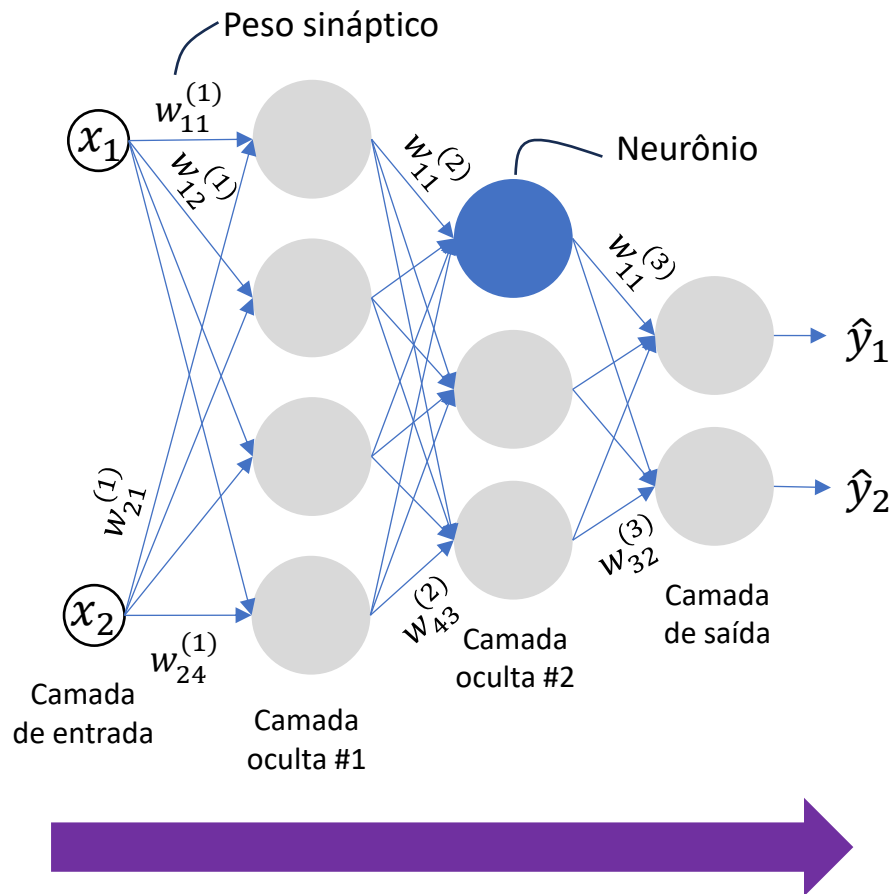


Redes neurais artificiais



- As MLPs também são chamadas de **redes densamente conectadas e de alimentação direta** (do inglês, *Dense Neural Network* - DNN).
 - Cada uma das saídas de uma camada **se conecta a todos os nós** da camada seguinte através de pesos sinápticos.
 - Os **dados fluem através da rede em uma única direção**, da camada de entrada para a camada de saída, **sem realimentação**.

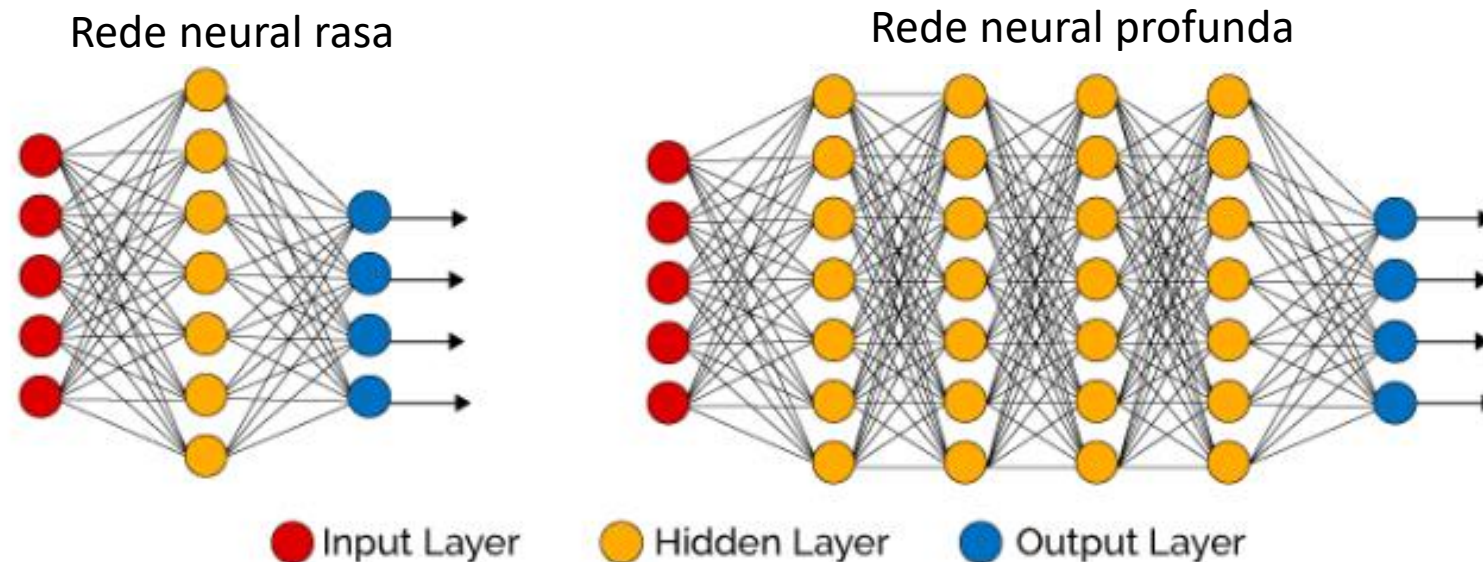
Redes neurais artificiais



- As **propriedades de uma rede neural** são determinadas por sua **arquitetura**.
 - Ela define o que a rede é capaz de aprender e quão bem ela fará isso.
- A arquitetura é definida
 - como os neurônios estão conectados (forma direta ou recursiva),
 - quantidade neurônios por camada,
 - quantidade de camadas escondidas,
 - funções de ativação,
 - etc.

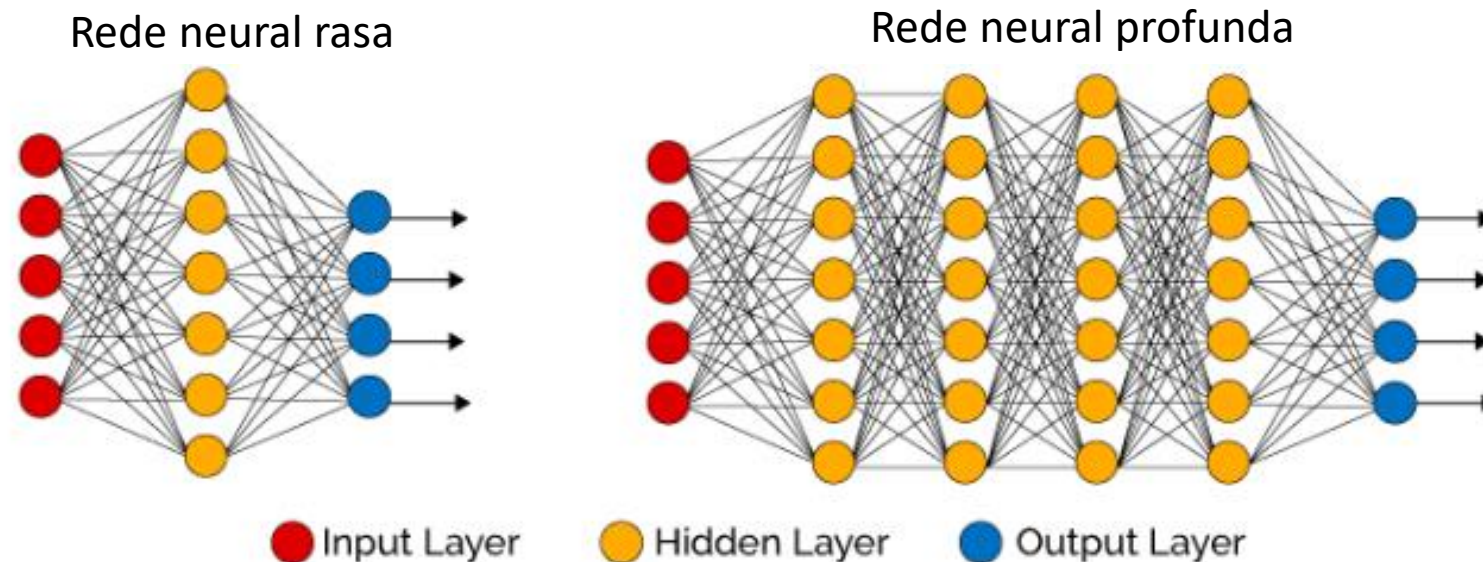
Redes neurais artificiais

- As RNAs são o coração do ***deep learning*** ou **aprendizado profundo**.
- O termo "**profundo**" vem fato destas redes poderem possuir **muitas camadas ocultas**.
- Quanto mais profunda, maior é a **capacidade de aprendizado** de uma rede neural.



Redes neurais artificiais

- Em geral, quando elas tem **duas ou mais camadas ocultas**, elas podem ser chamadas de **redes neurais profundas** (*Deep Neural Networks* - DNNs).
- Por terem uma grande capacidade de aprendizado, elas precisam de uma **grande quantidade de dados para aprenderem uma solução geral**.



Aproximação universal de funções

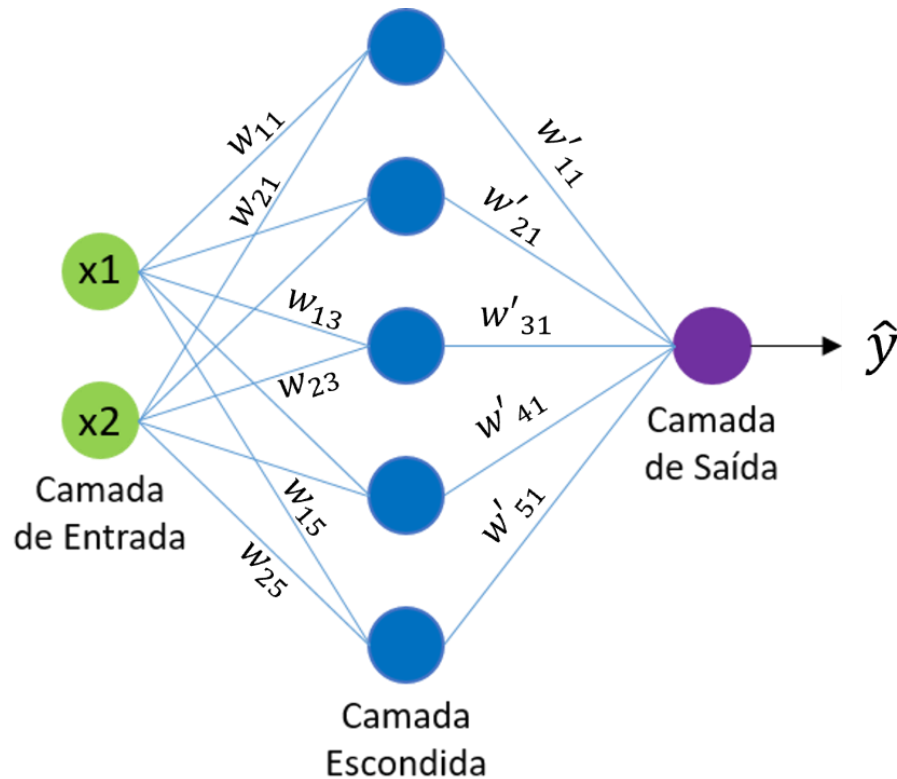
- As redes neurais têm a **capacidade de aproximar funções altamente não-lineares (e lineares também)**.
- Essa capacidade **depende da sua arquitetura**.
 - A **quantidade de pesos** de uma rede está associada aos seus **graus de liberdade**, ou seja, à **capacidade da rede de aproximar diferentes tipos de funções**.
 - Quanto maior a rede, mais pesos e, conseqüentemente, maior é sua capacidade de aprendizado.
- Portanto, assim como polinômios, que podem **aproximar qualquer tipo de função** (linear ou não linear) devido a seus **graus de liberdade**, as **redes neurais podem fazer o mesmo**, bastando apenas que **encontremos sua complexidade ideal**, ou seja, sua **arquitetura ideal**.

Aproximação universal de funções

- Por exemplo, uma rede neural com **uma camada oculta** com um número **suficiente de nós** pode **aproximar** praticamente qualquer **função contínua**.
- Com **duas camadas ocultas**, até **funções descontínuas** podem ser **aproximadas**.
- Portanto, dizemos que as redes neurais possuem **capacidade de aproximação universal** de funções.
- Desta forma, as redes neurais podem resolver **problemas de regressão e classificação** e uma grande gama de outros problemas.
- O desafio é encontrar a arquitetura e os pesos ideais para a aproximação.
 - Usamos **validação cruzada** e o algoritmo do **gradiente descendente**.

Como as redes neurais
aprendem?

Backpropagation

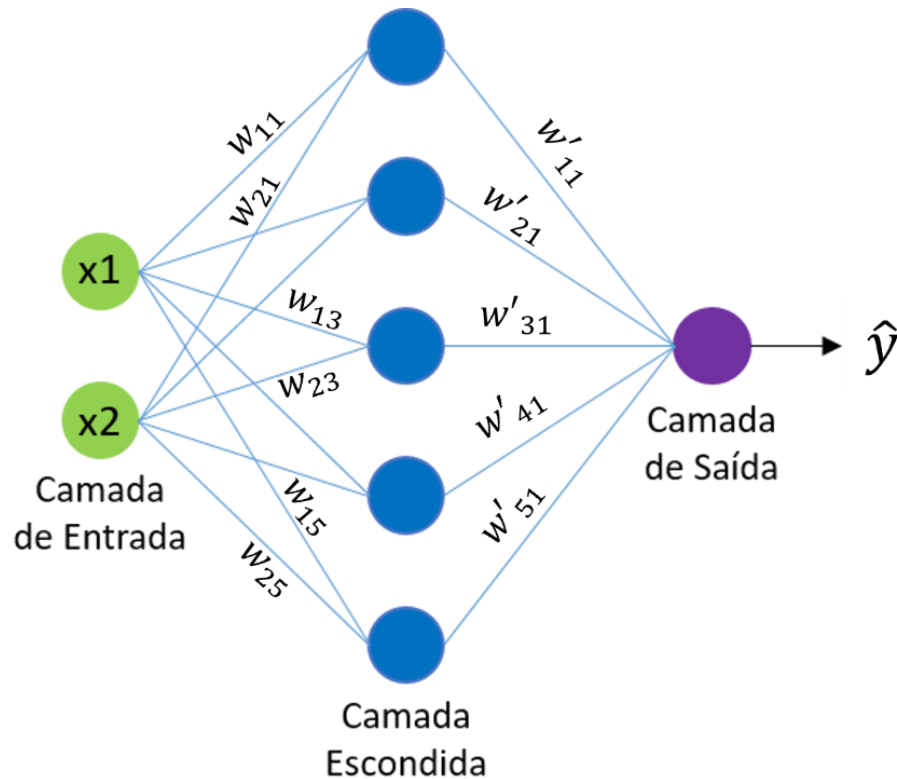


- As redes aprendem ajustando os pesos sinápticos e de *bias* para reduzir o erro.
- Isso é feito através de um algoritmo, baseado no gradiente descendente, chamado de (*error*) *backpropagation*.
- O **objetivo** do *backpropagation* é calcular o **quanto cada peso da rede contribui para o erro** e **ajustá-lo de forma a minimizar o erro**.



O algoritmo de *error backpropagation* é baseado na regra da cadeia.

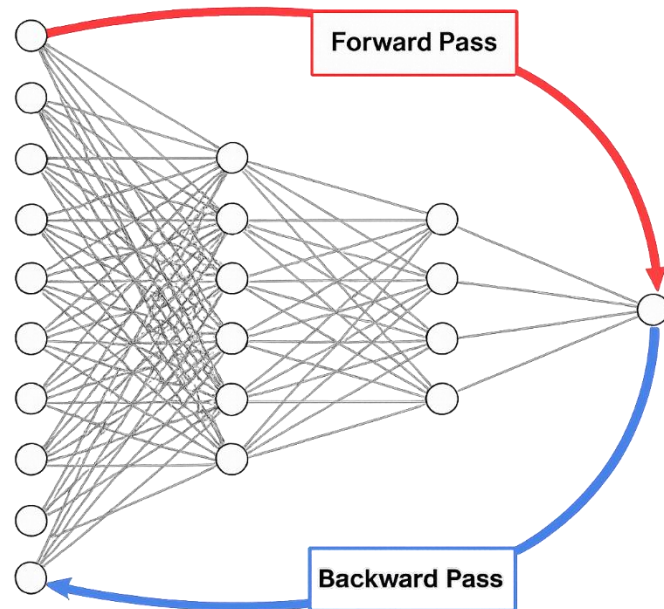
Backpropagation



- O algoritmo faz três coisas principais:
 - A rede faz previsões com os pesos atuais.
 - Etapa chamada de direta (*forward*).
 - Calcula o erro entre a saída da rede e os rótulos.
 - **Propaga o erro para trás** através das camadas da rede para atualizar os pesos usando o gradiente descendente.
 - Etapa chamada de retropropagação (*backpropagation*).
 - Os gradientes são calculados usando **regra da cadeia**.

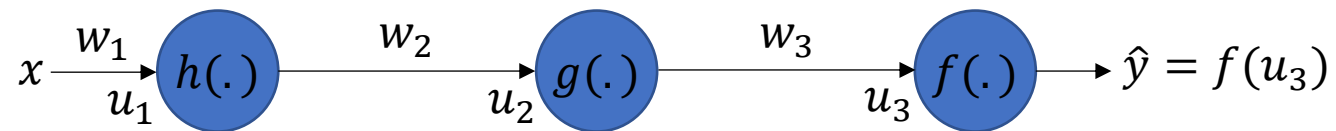
Backpropagation

- **Em resumo:** a rede prediz → mede o erro → corrige os pesos.
- O *backpropagation* responde à pergunta:
“Quais pesos e o quanto eles precisam mudar para que a rede produza uma saída mais correta?”



Regra da cadeia

- Para **atualizar o peso de um nó de uma camada**, a **retropropagação** calcula a derivada parcial do erro em relação àquele peso usando a **regra da cadeia**.
- Vejamos o exemplo abaixo com 3 nós, 3 pesos, w_1 , w_2 e w_3 , 3 funções de ativação, $f(\cdot)$, $g(\cdot)$ e $h(\cdot)$, uma entrada x , uma saída $\hat{y}$ e um rótulo y .

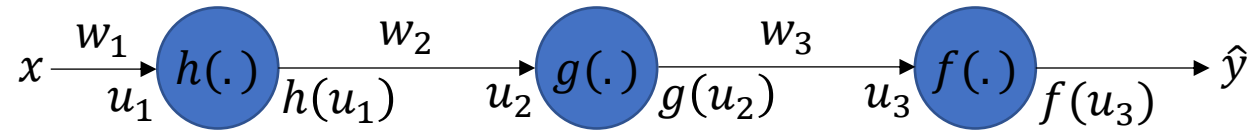


- Para facilitar a análise, vamos definir as entradas dos nós como

$$\begin{aligned}u_1 &= w_1 x, \\u_2 &= w_2 h(w_1 x) = w_2 h(u_1), \\u_3 &= w_3 g(w_2 h(w_1 x)) = w_3 g(u_2)\end{aligned}$$

- Essas variáveis são chamadas de **ativações**.

Regra da cadeia



- Qual é a derivada do erro $J_e = (y - \hat{y})^2$ em relação à w_1 ?

$$\frac{\partial J_e}{\partial w_1} = \frac{\partial (y - f(u_3))^2}{\partial f(u_3)} \frac{\partial f(u_3)}{\partial u_3} \frac{\partial u_3}{\partial g(u_2)} \frac{\partial g(u_2)}{\partial u_2} \frac{\partial u_2}{\partial h(u_1)} \frac{\partial h(u_1)}{\partial u_1} \frac{\partial u_1}{\partial w_1}.$$

- Calculando as derivadas, exceto as das funções de ativação, temos

$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

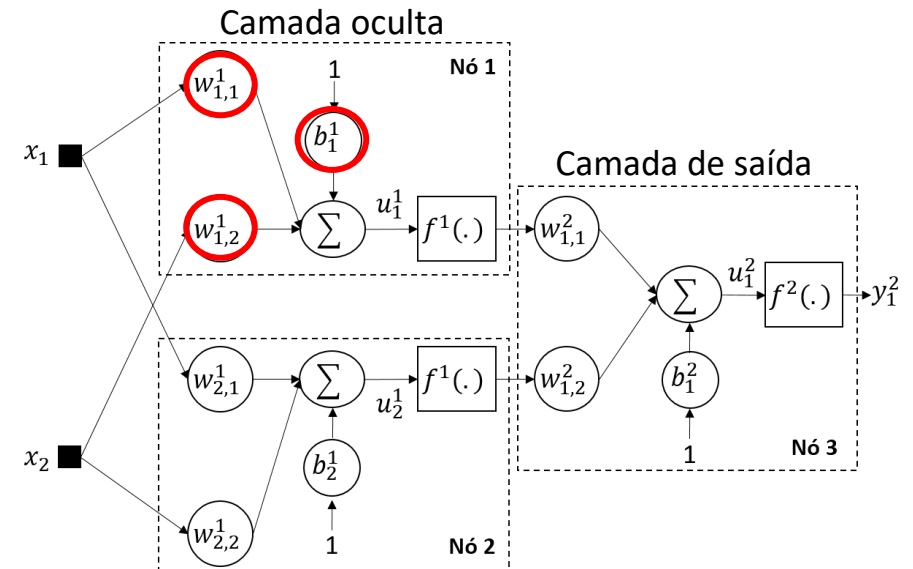
- **Exemplos:** derivadas das funções de ativação logística e tanh

- Logística: $f(x)(1 - f(x)) \in [0, 0.25]$
- Tanh: $1 - \tanh^2(x) \in [0, 1]$

Exemplo da aplicação da retropropagação

- Encontrar o **vetor gradiente** para todos os pesos do nó 1 da camada oculta da rede neural MLP abaixo.

$$\begin{bmatrix} \frac{\partial J_e}{\partial w_{1,1}^1} \\ \frac{\partial J_e}{\partial w_{1,2}^1} \\ \frac{\partial J_e}{\partial b_1^1} \end{bmatrix} = ?$$



- **OBS.:** vamos deixar as derivadas da função de ativação em relação às ativações, u , de forma genérica, i.e., não vamos assumir nenhum tipo específico de função de ativação.

Exemplo da aplicação da retropropagação

- Usaremos o **erro quadrático médio** como função de erro

$$\begin{aligned} J_e &= \frac{1}{N_{\text{dados}} N_M} \sum_{n=1}^{N_{\text{dados}}} \sum_{k=1}^{N_M} e_k^2(n) \\ &= \frac{1}{N_{\text{dados}} N_M} \sum_{n=1}^{N_{\text{dados}}} \sum_{k=1}^{N_M} \left(y_k(n) - \hat{y}_k^M(n) \right)^2, \end{aligned}$$

onde N_{dados} é o número de amostras, N_M é o número de nós na camada de saída, $y_k(n)$ e $\hat{y}_k^M(n)$ são o rótulo da k -ésima saída e a saída do k -ésimo nó da camada de saída, respectivamente, ambos correspondentes à n -ésima amostra de entrada, $\mathbf{x}(n)$ e M representa o índice da última camada.

Exemplo da aplicação da retropropagação

- A rede possui **uma camada oculta** com **dois nós** e **uma camada de saída** com **um único nó**, i.e., $N_M = 1$.
- Além disso, por motivos didáticos, vamos considerar um único exemplo de entrada, $\mathbf{x} = [x_1, x_2]$ e seu respectivo rótulo, y .
- Assim, $N_{\text{dados}} = 1$, fazendo com que o erro se torne

$$J_e = e_1^2 = (y - \hat{y}_1^2)^2,$$

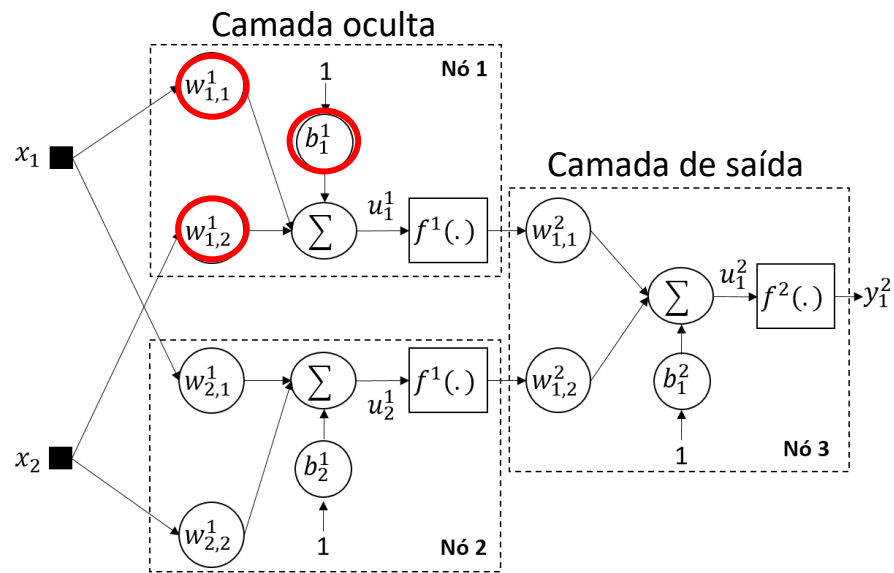
onde o índice n não é apresentado, pois temos apenas uma amostra.

- Por sua vez, o gradiente é dado por

$$\frac{\partial J_e}{\partial w_{i,j}^m} = \frac{\partial e_1^2}{\partial w_{i,j}^m}.$$

OBS.: $\hat{y}_1^2$ não significa “ao quadrado”, mas sim a indicação do índice da camada.

Exemplo da aplicação da retropropagação



- Vamos supor que os pesos de todos os nós são inicializados de forma aleatória.
- Assim, quando a entrada, $\mathbf{x}$, é apresentada à rede, é possível calcular todos os valores de interesse ao longo dela até sua saída.
- Consequentemente, tendo o valor de saída, conseguimos calcular o erro.
- Essa é a etapa **direta** (*forward*).

Exemplo da aplicação da retropropagação

- Aplicando a regra da cadeia, a derivada parcial do erro em relação ao peso $w_{1,1}^1$ é dada por

$$\frac{\partial J_e}{\partial w_{1,1}^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial w_{1,1}^1}.$$

- Resolvendo as derivadas parciais, temos

$$\frac{\partial J_e}{\partial w_{1,1}^1} = -2(y - \hat{y}_1^2) f'^2(u_1^2) w_{1,1}^2 f'^1(u_1^1) x_1,$$

onde $f'^m(u_i^m) = \frac{\partial f^m(u_i^m)}{\partial u_i^m}$ é a derivada parcial da função de ativação da m -ésima camada em relação à ativação do i -ésimo nó desta camada.

Exemplo da aplicação da retropropagação

- Aplicando-se o mesmo procedimento aos outros pesos, obtemos

$$\begin{aligned}\frac{\partial J_e}{\partial w_{1,1}^1} &= \frac{\partial e^2}{\partial w_{1,1}^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial w_{1,1}^1} \\ \frac{\partial J_e}{\partial w_{1,2}^1} &= \frac{\partial e^2}{\partial w_{1,2}^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial w_{1,2}^1} \\ \frac{\partial J_e}{\partial b_1^1} &= \frac{\partial e^2}{\partial b_1^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial b_1^1}\end{aligned}$$

Exemplo da aplicação da retropropagação

- Assim, o vetor gradiente com as derivadas parciais com relação a todos os pesos do **nó** $i = 1$ da camada $m = 1$ é apresentado abaixo

$$\begin{bmatrix} \frac{\partial J_e}{\partial w_{1,1}^1} \\ \frac{\partial J_e}{\partial w_{1,2}^1} \\ \frac{\partial J_e}{\partial b_1^1} \end{bmatrix} = -2(y - \hat{y}_1^2) f'^2(u_1^2) w_{1,1}^2 f'^1(u_1^1) \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix}.$$

Os pesos de **bias** estão ligados às entradas de todos os nós com valores constantes iguais a 1.

Pseudocódigo da retropropagação

- Inicialize a matriz de pesos, $\mathbf{W}$ (sinápticos e de *bias*) e o passo de aprendizagem, α .
- Faça $k = 0$ (contador de épocas)
- Calcule o erro inicial $J(\mathbf{W}(k))$ (opcional)
- Enquanto o critério de parada não for atendido, faça:
 - Para l variando de 1 até L , faça:
 - Apresente a l -ésima amostra, amostrada aleatoriamente sem reposição, à rede.
 - Calcule $\nabla J_l(\mathbf{W}(k))$
 - $\mathbf{W}(k + 1) = \mathbf{W}(k) - \frac{\alpha}{L} \sum_{l=1}^L \nabla J_l(\mathbf{W}(k))$
 - $k = k + 1$
 - Calcule $J(\mathbf{W}(k))$ (opcional)

onde $\nabla J_l(\mathbf{W}(k)) = \frac{1}{N_M} \sum_{j=1}^{N_M} \frac{\partial e_j^2(l)}{\partial \mathbf{W}(k)}$ é a média da derivada parcial do erro para todas as N_M saídas considerando apenas a l -ésima amostra.

OBS.: O pseudocódigo apresentado **implementa qualquer versão do gradiente descendente** variando o valor de L .

Vou ter que implementar tudo
isso manualmente?



What society thinks I do



What my friends think I do



What other computer scientists think I do



What mathematicians think I do



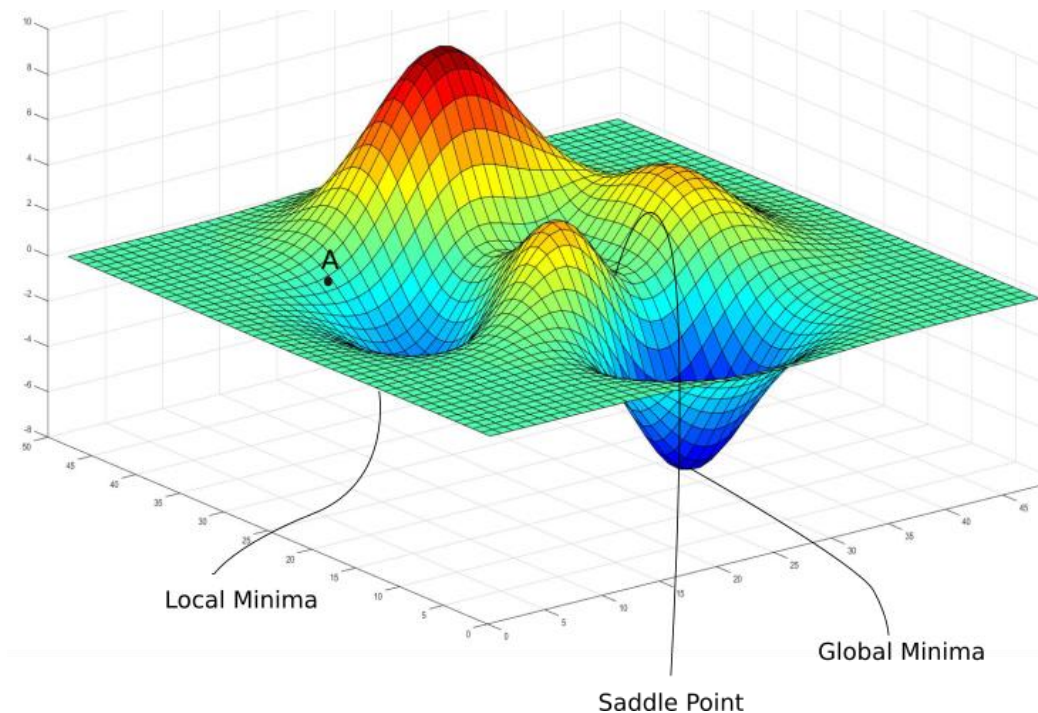
What I think I do

```
from tensorflow import *
```

What I actually do

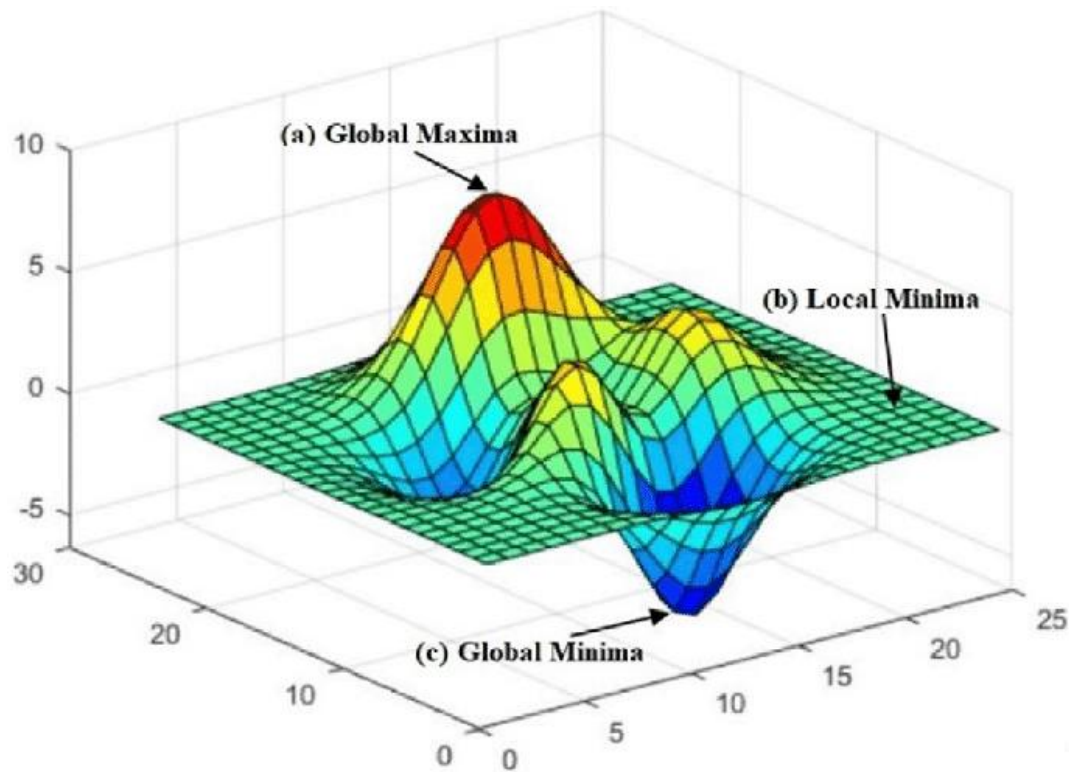
Não se preocupem, hoje em dia, não realizamos esse procedimento de forma manual, **usamos bibliotecas que automatizam o processo.**

Superfícies de erro de redes neurais



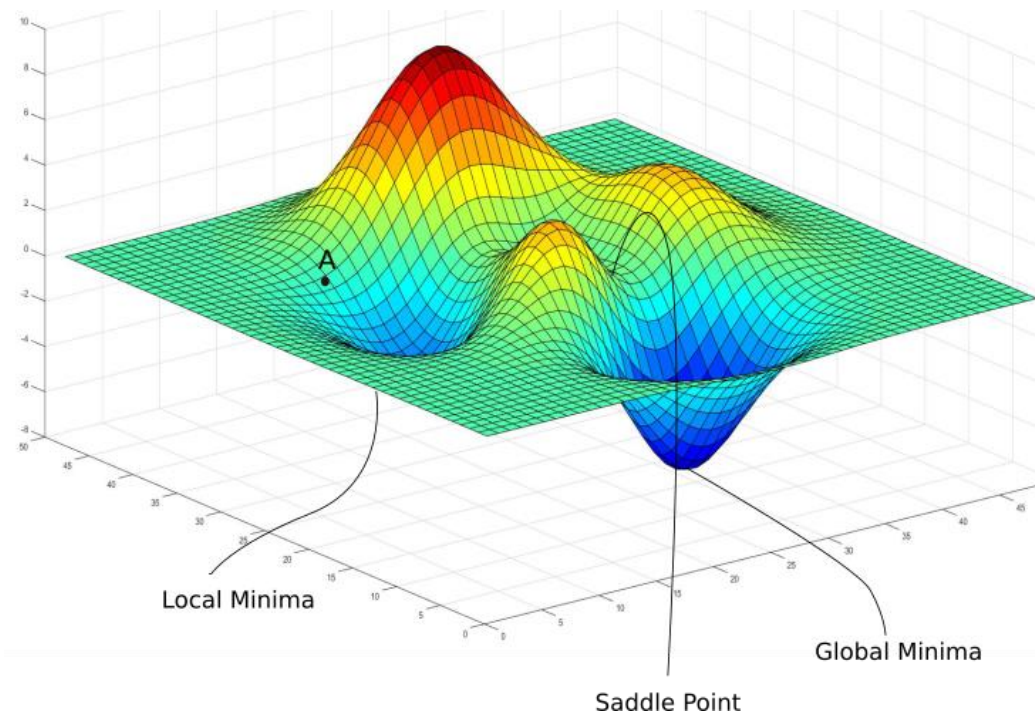
- Por serem formadas pela **combinação de vários nós com funções de ativação não-lineares**, as superfícies de erro de redes neurais **não são convexas**.
- Elas são **altamente irregulares**, incluindo irregularidades como
 - Pontos de sela: a função diminui em algumas direções e aumenta em outras.
 - Vários mínimos locais.
 - Platôs: região plana com erro alto.
 - etc.

Superfícies de erro irregulares



- Essas irregularidades causam
 - treinamento mais lento
 - instabilidade numérica
 - soluções diferentes para o mesmo problema
- Ou seja, as **irregularidades dificultam o aprendizado.**

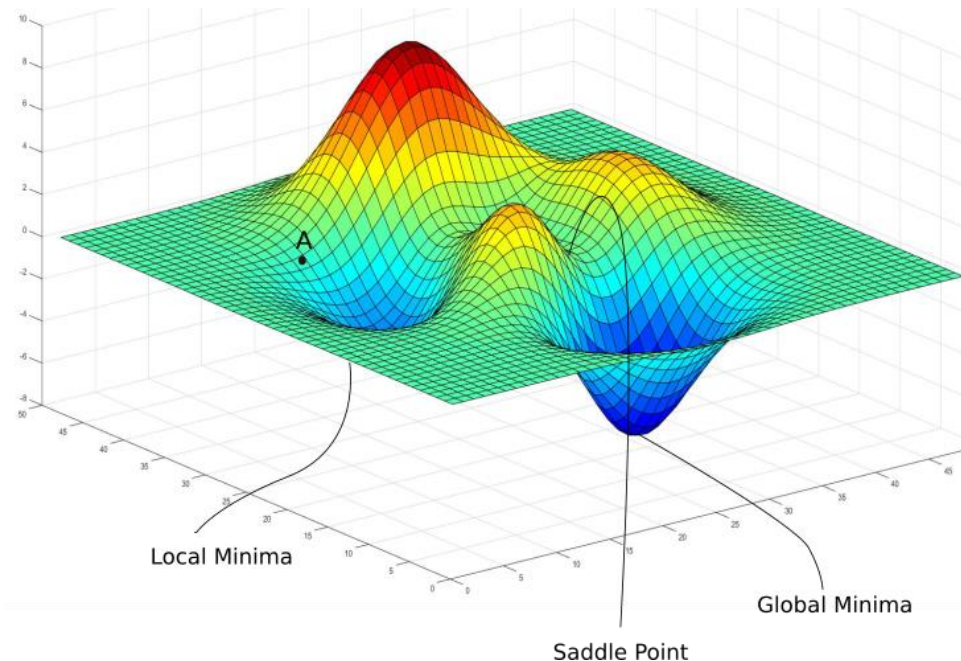
Superfícies de erro irregulares



- Felizmente, em geral, **quase todos os mínimos locais têm erro próximo ao do mínimo global** e, portanto, encontrar um desses mínimos locais já é bom o suficiente para um dado problema [1].
- Isso explica porque o **gradiente descendente funciona bem mesmo em problemas altamente não convexos**.

Mas como garantir que o valor de erro encontrado é bom o suficiente?

Como garantir que o erro encontrado é bom?



- Avaliar o erro em conjuntos de validação e teste.
 - O modelo é considerado bom se os **erros de validação e teste forem baixos**.
- Comparar o resultado de vários treinamentos com inicializações (pesos e *dataset*, e.g., *k-fold*) diferentes.
 - Se todos os treinamentos convergem para erros parecidos, então o mínimo ou os mínimos encontrados são provavelmente bons.
 - Se os resultados variam muito, então os mínimos podem ser ruins e isso pode indicar sobreajuste do modelo.

Exemplo

- [Exemplo: mlp.ipynb](#)



Lista de exercícios

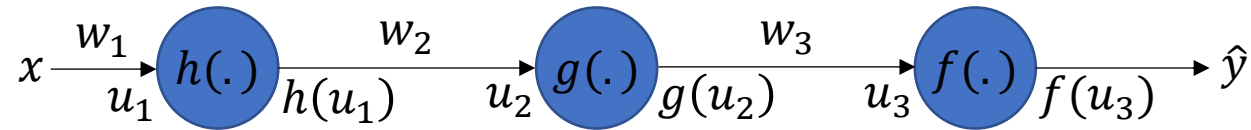
[Lista #9 - MLP](#)

Perguntas?

Obrigado!

Anexo I: Desaparecimento e explosão do gradiente

Desaparecimento do gradiente



- Vamos usar novamente a rede com **3 camadas** acima como exemplo.
- Como já vimos, derivada parcial do erro em relação ao peso w_1 do nó da primeira camada é dada por

$$\frac{\partial J_e}{\partial w_1} = \frac{\partial (y - f(u_3))^2}{\partial f(u_3)} \frac{\partial f(u_3)}{\partial u_3} \frac{\partial u_3}{\partial g(u_2)} \frac{\partial g(u_2)}{\partial u_2} \frac{\partial u_2}{\partial h(u_1)} \frac{\partial h(u_1)}{\partial u_1} \frac{\partial u_1}{\partial w_1}.$$

- Podemos reescrevê-la deixando apenas as derivadas das funções de ativação de forma genérica

$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

Desaparecimento do gradiente

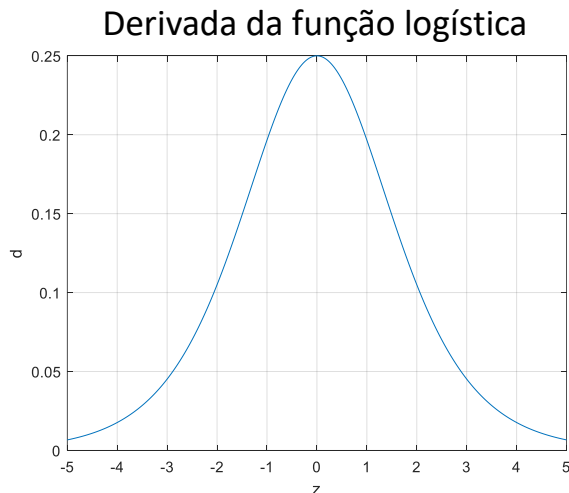
$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

- A rede com 3 camadas tem 3 derivadas parciais das funções de ativação para a atualização do peso do nó da primeira camada.
- Se extrapolarmos a equação acima para redes com M camadas, teremos a multiplicação de M derivadas das funções de ativação para a derivada parcial do erro em relação ao peso da primeira camada.

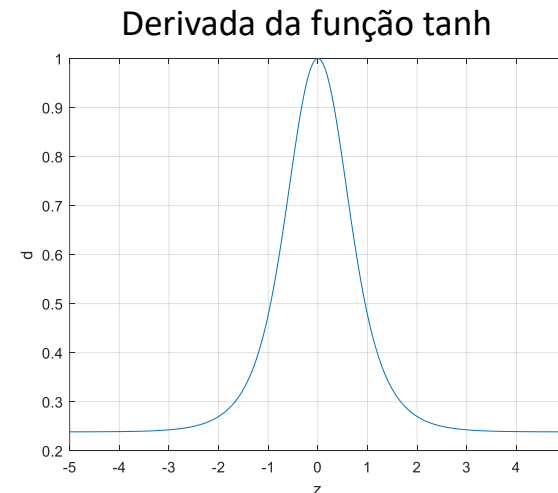
Desaparecimento do gradiente

$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

- As derivadas das funções logística e tangente hiperbólica têm valores máximos iguais a 0.25 e 1, respectivamente, apenas quando as ativações são iguais a 0.



$$f(z)(1 - f(z)) \in [0, 0.25]$$



$$1 - \tanh^2(z) \in [0, 1]$$

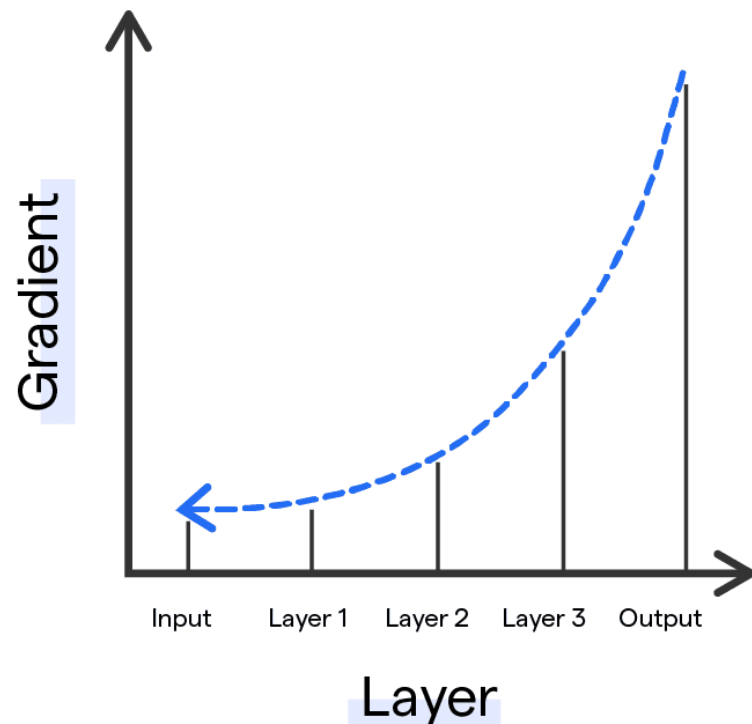
Desaparecimento do gradiente

- Se usarmos a função logística, teremos a multiplicação de vários termos menores que 0 (no máximo iguais a 0.25 quando a entrada é igual a 0).
- Usando a função tanh, teremos a multiplicação de vários termos com valores no máximo iguais a 1, mas apenas quando a entrada é igual a 0.
- Assim, em uma rede com M camadas, a **retropropagação** tem o efeito de multiplicar até M valores, em geral, menores do que 1 para calcular os vetores gradiente das primeiras camadas.
- **OBS.:** Se os valores iniciais dos pesos forem menores do que 1, também teremos a multiplicação de vários termos pequenos.

Mas o que isso significa na prática?

Desaparecimento do gradiente

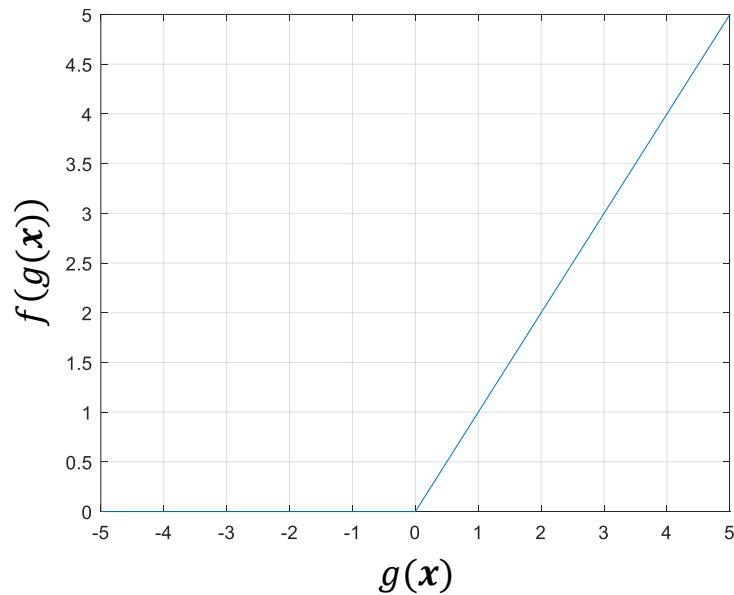
Vanishing Gradient Problem



- Isso significa que o **gradiente diminui exponencialmente com M** .
- Assim, os **nós das camadas iniciais aprendem muito mais lentamente** do que os nós das camadas finais, pois o **vetor gradiente** daquelas camadas é **muito pequeno**, fazendo com que a **atualização dos pesos também seja pequena**.
- Isso faz com que as **primeiras camadas aprendam lentamente ou nem aprendam**, pois têm **derivadas muito pequenas, tendendo a zero**.

Como mitigar esse problema?

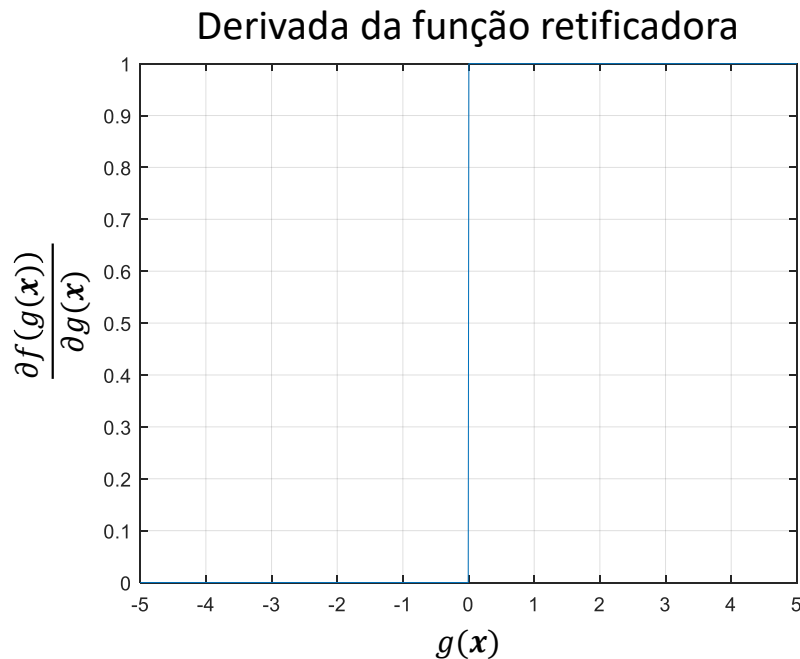
Função de ativação retificadora



$$\hat{y} = f(g(x)) = \max(0, g(x))$$

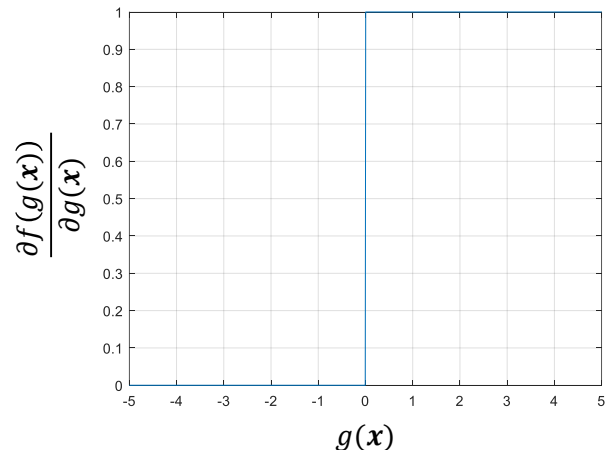
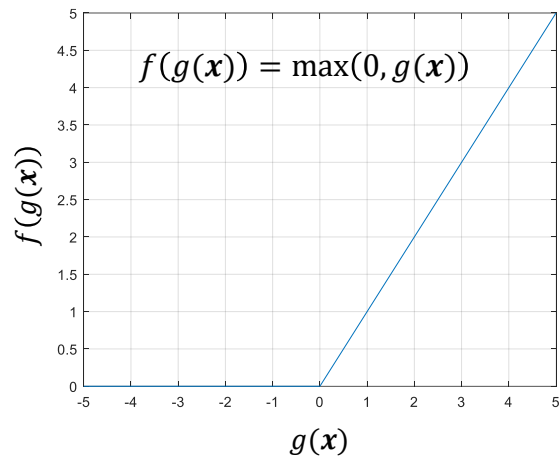
- Com o surgimento das **redes neurais profundas**, e, consequentemente, do problema do **desaparecimento do gradiente**, uma outra função de ativação, conhecida como ***Rectified Linear Unit*** (ReLU), passou a ser bastante utilizada.
- É também uma ***função não-linear*** onde sua saída é igual 0 quando $g(x) \leq 0$ e o próprio $g(x)$ quando $g(x) > 0$.
- É uma das funções mais utilizadas em redes neurais profundas atualmente.

Função de ativação retificadora



- Suas principais **vantagens** são a sua **simplicidade e eficiência computacional**.
 - Ela e sua derivada **são mais rápidas de se calcular** do que as funções logística e tangente hiperbólica.
- Além disso, ajuda a **minimizar o problema do desaparecimento de gradiente**, pois sua derivada é igual a 1 para $g(\mathbf{x}) > 0$.
- Sua derivada é dada por
$$\frac{dy_j}{dg(\mathbf{x})} = \frac{df(g(\mathbf{x}))}{dg(\mathbf{x})} = \begin{cases} 0, & \text{se } g(\mathbf{x}) < 0 \\ 1, & \text{se } g(\mathbf{x}) > 0 \end{cases}$$
- A derivada é indeterminada para $g(\mathbf{x}) = 0$.

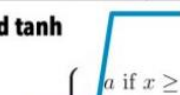
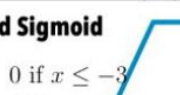
Função de ativação retificadora



- Uma desvantagem é que ela causa o problema conhecido como **ReLU agonizante**.
- Esse **problema ocorre durante o treinamento** da rede, quando a ativação do nó, $g(x)$, é **negativa**.
- Isso faz com que sua **saída e**, consequentemente, a **derivada parcial da função de ativação sejam iguais a 0**.
- Quando isso ocorre, o **nó não tem seus pesos atualizados** durante o treinamento, **permanecendo inalterados**.

Variantes da função de ativação retificadora

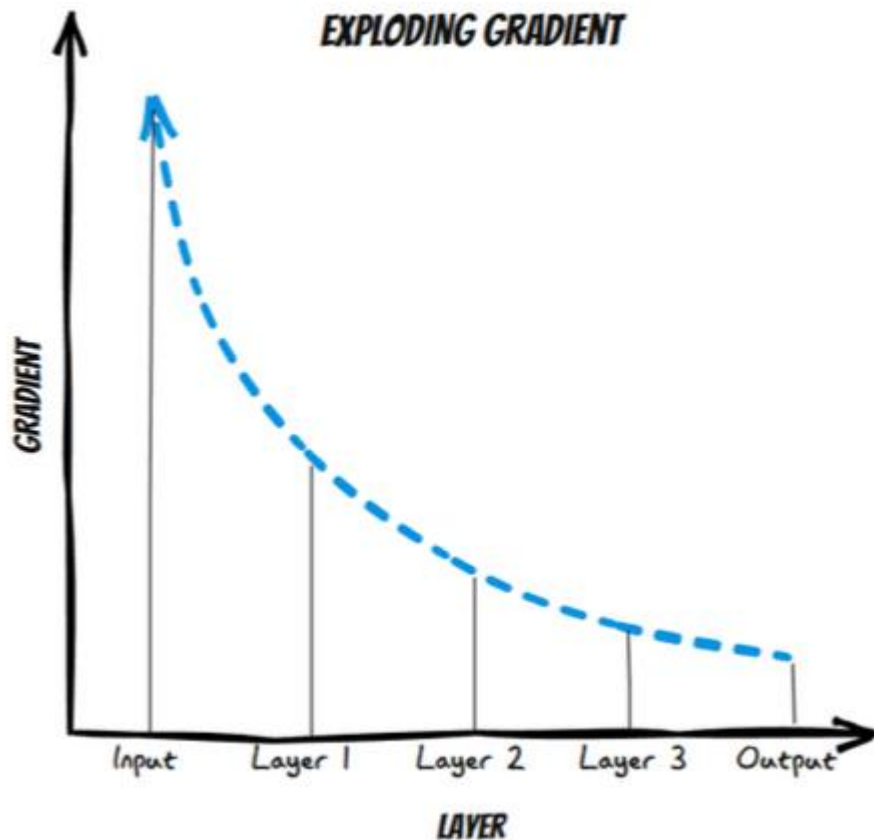
Neural Network Activation Functions: a small subset!

ReLU  $\max(0, x)$	GELU  $\frac{1}{\sqrt{2\pi}} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

- Para mitigar o problema das **ReLU**s agonizantes, usa-se **variantes da função ReLU** que **possuam derivada diferente de zero para $g(x) < 0$** , como, por exemplo,

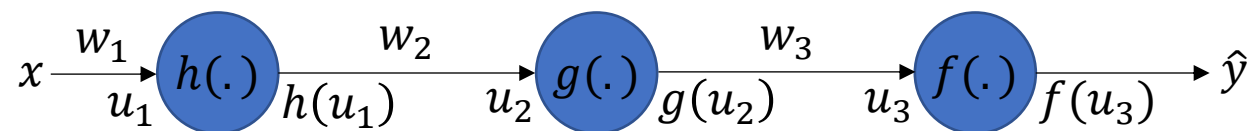
- [Leaky ReLU](#),
- [Parametric ReLU \(PReLU\)](#),
- [Gaussian Error Linear Unit \(GELU\)](#),
- [etc.](#)

Explosão do gradiente



- Usando funções de ativação ReLU e suas variantes, reduzimos o problema do desaparecimento do gradiente.
- Porém, um **outro problema surge** quando as **ativações são positivas (derivadas iguais a 1)** e os **pesos têm valores maiores do que 1**.
- Caso os pesos sejam inicializados (em geral, de forma aleatória) com **valores maiores do que 1**, haverá a **multiplicação de vários valores assim**, o que pode resultar em **valores de gradiente muito grandes nas camadas iniciais**.

Explosão do gradiente



$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

- Se os elementos do vetor gradiente tiverem **magnitudes muito grandes**, os **pesos da rede podem sofrer atualizações extremamente grandes**, o que leva a **instabilidades numéricas** e a um **treinamento ineficaz** ou até mesmo à **divergência**.

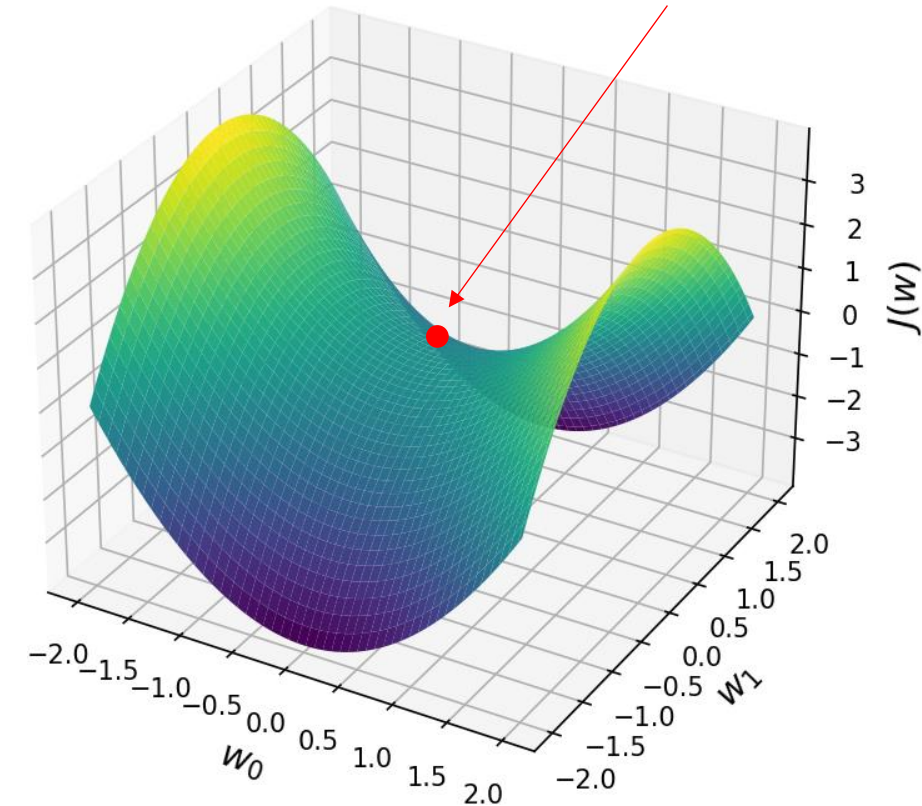
Formas de se minimizar a dissipação e a explosão do gradiente

- Além do uso de funções de ativação ReLU ou de suas variantes, outras formas de se minimizar esses problemas são:
 - **Inicialização apropriada dos pesos:** garante que a variância das ativações permaneça a mesma ao longo de todas as camadas. Isso garante que o gradiente retropropagado não tenha multiplicações com valores muito pequenos ou muito grandes em qualquer camada, ajudando a mitigar ambos os problemas.
 - **Normalização das ativações:** padroniza as ativações das camadas durante o treinamento. Isso estabiliza os gradientes e acelera a convergência.
 - **Poda do gradiente:** limita o valor máximo do gradiente durante o treinamento. Isso evita atualizações excessivas dos pesos e previne a explosão do gradiente.
 - **Arquiteturas com conexões residuais:** criam caminhos diretos entre camadas. Isso facilita a propagação do gradiente em redes muito profundas.

Anexo II: Algumas irregularidades que dificultam o aprendizado de redes neurais

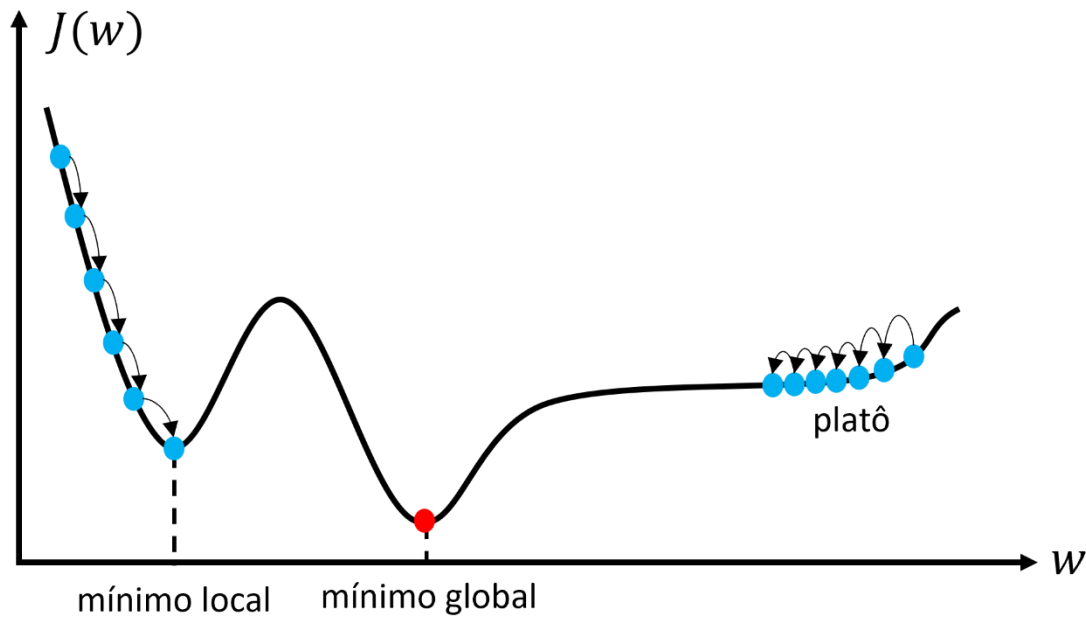
Pontos de sela

Superfície com Ponto de Sela



- Uma irregularidade comumente encontrada em redes neurais são os **pontos de sela**:
 - É um ponto que é um **mínimo ao longo de um eixo**, mas um **máximo ao longo de outro**.
 - Em algumas direções são **atratores** (i.e., alta declividade), mas em outras não.
- O algoritmo de otimização pode passar um longo período de tempo sendo atraído por eles, o que prejudica seu desempenho.
- Para escapar destes pontos, usa-se métodos de **segunda ordem** ou **versões estocásticas (i.e., ruidosas)** do gradiente descendente.

Platôs



- Outro tipo de irregularidade são os **platôs**.
- Eles são **regiões planas e com erro elevado**.
- Como a **inclinação da superfície** nessa região é **próxima de zero** (i.e., o gradiente é próximo de zero) o algoritmo pode levar muito tempo para atravessá-la.
- Métodos de **aprendizado adaptativo**, que são variações do gradiente descendente, como AdaGrad, RMSProp, Adam, podem escapar destas regiões.